



LAB MANUAL

SPEECH SIGNAL PROCESSING

CSSP-469L

Dr. Sajjad A. Ghauri

OBJECTIVES:

Synthesize block diagrams for speech applications, explain the purpose of various blocks, and describe in detail algorithms that can be used to implement them. Implement components of speech processing systems, including speech recognition and speaker recognition, using MATLAB.

INTRODUCTION: The Speech Processing by Computer course provides a foundational introduction to the concepts, methods, and applications of speech signal processing. It covers key principles of digital processing, including the acquisition and playback of signals, the operation of linear digital systems, spectral analysis, fundamental frequency analysis, and formant analysis. Additionally, the course explores applications in speech synthesis and speech recognition. Lecture materials are integrated with laboratory sessions that utilize pre-existing application programs. [captionList of Labs for Speech Signal Processing](#)

SOFTWARE TOOLS: MATLAB

LAB OUTLINE

Lab No.	Lab Title
Lab 1	Signal Processing in MATLAB
Lab 2	Introduction to Discrete Time Signal Processing in MATLAB
Lab 3	Analysis of Z-Transform in MATLAB
Lab 4	Study of Convolution and Discrete Fourier Transform
Lab 5	FIR Filter Design using MATLAB
Lab 6	Filter Design using FDA TOOL (FIR & IIR)
Lab 7	Cepstrum Analysis and Homomorphic Deconvolution
Lab 8	Speech Signal Classification using Different Neural Network Algorithms
Lab 9	Speech Synthesis
Lab 10	Open Ended Lab
Lab 11	Mel-Frequency Cepstral Coefficients (MFCCs) for Speech Recognition
Lab 12	Speech Recognition using Mel-Frequency Cepstral Coefficients (MFCCs)
Lab 13	Speech Recognition using Mel-Frequency Cepstral Coefficients (MFCCs) – Part B
Lab 14	Speech Classification using Various Machine Learning Algorithms

Lab 1

Signal Processing in MATLAB

September 18, 2025

1 Objective

- Introduction to digital signal processing and its applications.
- To study different signals and data types in MATLAB.

The purpose of this lab is to understand the fundamental steps involved in digital signal processing, which include signal generation, sampling, quantization, encoding, and reconstruction of quantized signals. This lab covered the essential processes in digital signal processing.

- Reading & Plotting a Signal
- Sampling the Signal at a Lower Frequency
- Quantizing the Signal to 8-bit Levels
- Encoding the Signal into Binary Format
- Saving the Processed Audio File

Each step demonstrated the effects of digitizing an analog signal for digital transmission and storage.

2 Task 1: Signal Generation (.wav File)

To read an audio signal from a .wav file and visualize its waveform.

2.1 Steps

1. Load an audio signal using the `audioread()` function.
2. Extract the original sampling frequency F_s .
3. Plot the original signal waveform to observe its amplitude variations over time.

3 Task 2: Sampling of the Signal

To downsample the original signal to a lower frequency (e.g., 8 kHz) to reduce the data rate while preserving essential information.

3.1 Steps

1. Define a new sampling frequency $F_{s_{new}} = 8000$ Hz.
2. Resample the signal using the `resample()` function.
3. Plot the downsampled signal.

4 Task 3: Quantization of the Signal

To quantize the sampled signal into discrete levels based on an 8-bit quantization scheme.

4.1 Steps

1. Define the number of bits $nbits = 8$.
2. Calculate the number of quantization levels $L = 2^{nbits}$.
3. Determine the minimum (x_{min}) and maximum (x_{max}) amplitude values of the signal.
4. Compute the quantization step size $\Delta = \frac{x_{max} - x_{min}}{L - 1}$.
5. Apply uniform quantization.
6. Plot the quantized signal.

5 Task 4: Encoding of the Signal

To convert the quantized values into a binary representation for digital storage or transmission.

5.1 Steps

1. Convert each quantized sample into its corresponding binary form.
2. Use the `dec2bin()` function to represent each sample using $nbits$ bits.
3. Display the first few binary-encoded samples.

6 Task 5: Saving the Quantized Signal

To save the quantized speech signal as a .wav file for future use or analysis.

6.1 Steps

1. Use the `audiowrite()` function to save the quantized signal.
2. Specify the new sampling frequency Fs_{new} .

7 Observations & Discussion

- **Effect of Sampling:** Compare the original and sampled signals to observe the impact of reducing the sampling rate. Higher sampling rates preserve more details, while lower rates may cause loss of high-frequency components.
- **Effect of Quantization:** Observe the difference between the original and quantized signals. Increasing the number of bits ($nbits$) improves the signal quality but increases data size.
- **Encoding Process:** Check how signal amplitudes are mapped to binary values. The binary representation is necessary for digital communication and storage.

Lab 2

Introduction to Discrete Time Signal Processing in MATLAB

September 25, 2025

1 Objective

- Introduction to digital signal processing and its applications.
- To study different signals and data types in MATLAB.

2 Theory

Digital signal processing involves the processing of a discrete-time signal (input signal) to produce another discrete-time signal (output signal) which has some desired properties. A strong background in discrete-time signals is essential for analyzing discrete-time systems. This lab focuses on the creation of discrete-time signals in MATLAB and explains the required MATLAB commands. After performing the lab, the students will be able to:

- Create discrete-time signals in MATLAB.
- Use MATLAB commands to perform simple operations on discrete-time signals.

3 Generation of Sequences

Following are some of the commonly used discrete-time signals generated using MATLAB:

3.1 Unit Impulse

A unit impulse sequence of length N can be generated in MATLAB by the following command:

```
1 n = -10:10;
2 impulse = [zeros (1,10) 1 zeros (1,20)];
3 stem(n, impulse);
4 title('Unit Impulse Sequence');
5 xlabel('n'); ylabel('Amplitude');
6 grid on;
```

3.2 Delayed Unit Impulse

A unit impulse sequence delayed by M samples where $M < N$ can be generated as follows:

```
1 n = -10:10;
2 delayed_impulse = (n == M);
3 stem(n, delayed_impulse);
4 title('Delayed Unit Impulse Sequence');
5 xlabel('n'); ylabel('Amplitude');
6 grid on;
```

3.3 Unit Step

Likewise, a unit step sequence $s[n]$ of length N can be generated using the MATLAB command:

```
1 n = -10:10;
2 unit_step = (n >= 0);
3 stem(n, unit_step);
4 title('Unit Step Sequence');
5 xlabel('n'); ylabel('Amplitude');
6 grid on;
```

A delayed unit step sequence can be generated in a manner similar to that used in the generation of a delayed unit sample sequence.

4 Sinusoidal Sequences

Sinusoidal sequences can be generated in MATLAB using `sin` and `cos` functions. The MATLAB code that generates a sinusoidal signal:-

```
1 n = 0:20;
2 x = sin(2*pi*0.1*n);
3 stem(n, x);
4 title('Sinusoidal Sequence');
5 xlabel('n'); ylabel('Amplitude');
```

```
6 grid on;
```

5 Linear and Nonlinear Systems

We now investigate the linearity property of a causal system. Consider the system given by:

$$y[n] - 0.4y[n-1] + 0.75y[n-2] = 2.2403x[n] + 2.4908x[n-1] + 2.2403x[n-2]$$

5.1 MATLAB Implementation

```
1 % Generate the input sequences
2 close all;
3 clear all;
4 clc
5
6 n = 0:40;
7 a = 2;
8 b = -3;
9 x1 = cos(2*pi*0.1*n);
10 x2 = cos(2*pi*0.4*n);
11 x = a*x1 + b*x2;
12
13 num = [2.2403 2.4908 2.2403];
14 den = [1 -0.4 0.75];
15 ic = [0 0]; % Set zero initial conditions
16
17 y1 = filter(num,den,x1,ic); % Compute the output y1[n]
18 y2 = filter(num,den,x2,ic); % Compute the output y2[n]
19 y = filter(num,den,x,ic); % Compute the output y[n]
20 yt = a*y1 + b*y2;
21 d = y - yt; % Compute the difference output d[n]
22
23 % Plot the outputs and the difference signal
```

```

24 subplot(3,1,1)
25 stem(n,y);
26 ylabel('Amplitude');
27 title('Output Due to Weighted Input');
28
29 subplot(3,1,2)
30 stem(n,yt);
31 ylabel('Amplitude');
32 title('Weighted Output');
33
34 subplot(3,1,3)
35 stem(n,d);
36 xlabel('Time index n');
37 ylabel('Amplitude');
38 title('Difference Signal');

```

6 Time-Invariant and Time-Varying Systems

We next investigate the time-invariance property of a causal system. MATLAB Program P1_3 is used to simulate the system, generate two different input sequences $x[n]$ and $x[n - D]$, and compute and plot the corresponding output sequences $y_1[n]$, $y_2[n]$, and the difference $y_1[n] - y_2[n + D]$.

6.1 MATLAB Implementation

```

1 close all; clear all; clc
2 n = 0:40; D = 10; a = 3.0; b = -2;
3 x = a*cos(2*pi*0.1*n) + b*cos(2*pi*0.4*n); xd = [zeros(1,D) x];
4 num = [2.2403 2.4908 2.2403];
5 den = [1 -0.4 0.75];
6 ic = [0 0]; % Set initial conditions
7 % Compute the output y[n]
8 y = filter(num,den,x,ic);
9 % Compute the output yd[n]

```

```

10 yd = filter(num,den,xd,ic);
11 % Compute the difference output d[n]
12 d=y- yd(1:D:41+D);
13 % Plot the outputs subplot(3,1,1) stem(n,y); ylabel('Amplitude');
14 title('Output y[n]');
15 grid;
16 subplot(3,1,2)
17 stem(n,yd(1:41));
18 ylabel('Amplitude');
19 title(['Output Due to Delayed Input]);
20 grid;
21 subplot(3,1,3)
22 stem(n,d);
23 xlabel('Time index n');
24 ylabel('Amplitude');
25 title('Difference Signal');
26 grid;

```

7 Linear Time-Invariant Discrete-Time Systems

The MATLAB command `y = impz(num,den,N)` can be used to calculate the first N samples of the impulse response of the causal LTI discrete-time system.

```

1 % Compute the impulse response y clf;
2 N = 40;
3 num = [2.2403 2.4908 2.2403];
4 den = [1 -0.4 0.75];
5 y = impz(num,den,N);
6 % Plot the impulse response stem(y);
7 xlabel( Time index n );
8 ylabel( Amplitude );
9 title( Impulse Response );
10 grid;

```

8 Lab Task

1. If the LTI system changes from $y[n] = 0.5(x[n] + x[n-1])$ to $y[n] = 0.5(x[n] - x[n-1])$, what would its effect on the input $x[n] = s_1[n] + s_2[n]$?
2. Assess all tasks and provide a logical explanation for each using graphs.

9 Conclusion

Discrete-time signals and operations have been successfully verified in MATLAB.

Lab 3

Analysis of Z-Transform in Matlab

October 2, 2025

1 Objective:

1. Study Inverse and Wiener Filtering.
2. Study different types of noises.

2 Software Required:

- Computer workstation
- Matlab 2015 or above

3 Theory:

Description: In mathematics and signal processing, the Z-transform converts a discrete time-domain signal, which is a sequence of real or complex numbers, into a complex frequency-domain representation. The Z-transform, like many other integral transforms, can be defined as either a one-sided or two-sided transform.

3.1 Bilateral Z-transform:

The bilateral or two-sided Z-transform of a discrete-time signal $x[n]$ is the function $X(z)$ defined as

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]z^{-n} \quad (1)$$

3.2 Unilateral Z-transform:

Alternatively, in cases where $x[n]$ is defined only for $n \geq 0$, the single-sided or unilateral Z-transform is defined as:

$$X(z) = \sum_{n=0}^{\infty} x[n]z^{-n} \quad (2)$$

In signal processing, this definition is used when the signal is causal.

3.2.1 Rational Z-transform to Factored Z-transform:

Let the given transfer function be in the rational form,

$$G(z) = \frac{2z^4 + 16z^3 + 44z^2 + 56z + 32}{3z^4 + 3z^3 - 15z^2 + 18z - 12} \quad (3)$$

It is required to convert it into factored form, so that we can find the poles and zeros mathematically by applying the quadratic equation. Matlab command required for converting rational form to factored form is `zp2sos`. The factored form of $G(z)$ as evaluated by `zp2sos` is:

3.2.2 Factored Z-transform / Zeros, Poles to Rational Z-transform:

It is the inverse of the above case. When the transfer function is given in factored form and one is required to convert it into rational form, a single Matlab command can serve the purpose. Let us use the above result, that is, transfer function in factored form:

$$G(z) = \frac{(0.6667 + 0.4z^{-1} + 0.5333z^{-2})(1.000 + 2.000z^{-1} + 2.000z^{-2})}{(1.000 + 2.000z^{-1} - 4.000z^{-2})(1.000 - 1.000z^{-1} + 1.000z^{-2})} \quad (4)$$

For building up the transfer function in rational form, we find the poles and zeros of the above system simply by using the Matlab `roots` command or by hand. If we already have the poles and zeros of the given system, we can find the transfer function in factored form. The Matlab command that converts poles and zeros of the system into transfer function is `zp2tf`.

3.2.3 Rational Z-transform to Partial Fraction Form:

This technique is usually used while taking the inverse Z-transform and when the order of $H(z)$ is high, making it quite difficult to solve mathematically. Consider the transfer function in the rational form:

$$G(z) = \frac{18z^3}{18z^3 + 3z^2 - 4z - 1} \quad (5)$$

We can evaluate the partial fraction form of the above system using Matlab command. The Matlab command that converts rational Z-transform into partial fraction form is

residuez.

$$G(z) = \frac{0.36}{1 - 0.5z^{-1}} + \frac{0.24}{1 + 0.33z^{-1}} + \frac{0.4}{1 + 0.33z^{-1}} \quad (6)$$

3.2.4 Zplane: Zero-Pole Plot

This function displays the poles and zeros of discrete-time systems; **zplane(b,a)**

1. **pzmap**: Computes pole-zero map of LTI models. **Syntax**: pzmap(sys)
2. **ztrans**: Computes z-transform of function. **Syntax**: F=ztrans(f)

Matlab Code:

Z-transform:

```
syms z n
a = ztrans(1/1^n)
syms n
f = n^4;
ztrans(f)
```

Result is:

$$\frac{z(z^3 + 11z^2 + 11z + 1)}{(z - 1)^5} \quad (7)$$

Inverse Z-transform:

```
syms Z n
a = iztrans(3Z/(Z+1))
syms Z
f = 2Z/(Z-2)^2;
iztrans(f)
```

Result is:

$$2^n + 2^n(n - 1) \quad (8)$$

Example:

```
b = [0 1 1];
a = [1 -2 3];
```

```
roots(a);
roots(b);
zplane(b,a);
```

The **freqz** function computes and displays the frequency response of a given Z-transform function:

```
freqz(b, a, Fs)
```

The **impz** function computes the impulse response of the system:

```
impz(Num, Den)
```

3.3 Lab Task:

1. Express the following Z-transform in factored form, plot its poles and zeros, and then determine its ROC:

$$G(z) = \frac{2z^4 + 16z^3 + 44z^2 + 56z + 32}{3z^4 + 3z^3 - 15z^2 + 18z - 12} \quad (9)$$

3.4 Lab Assignment:

Prepare a lab report on Overleaf, including detailed analysis and explanations of graphs, along with the evaluated answers.

3.5 Conclusion:

Z-transform analysis has been verified successfully.

Lab 4

Study of convolution and Discrete Fourier transform

October 9, 2025

1 Theory

The output of an LTI system $y(n)$ can be computed by convolving the input signal $x(n)$ with the system response $h(n)$. A fast convolution method includes the FFT of the signal and system response. For longer convolutions, the savings become significant compared to direct convolution. Direct convolution requires $O(N^2)$ operations, while FFT-based convolution requires $O(N \log N)$ operations.

1.1 MATLAB Code for Convolution

```
1 close all;
2 clear all;
3 x=input('Enter x: ');
4 h=input('Enter h: ');
5 m=length(x);
6 n=length(h);
7 X=[x,zeros(1,n)];
8 H=[h,zeros(1,m)];
9 for i=1:n+m-1
10     Y(i)=0;
11     for j=1:m
12         if(i-j+1>0)
13             Y(i)=Y(i)+X(j)*H(i-j+1);
14         end
15     end
16 end
17 Y
18 stem(Y);
19 ylabel('Y[n]');
20 xlabel('n');
21 title('Convolution of Two Signals without conv function');
```

2 Definition of DFT and IDFT

The Fourier transform maps a time-domain signal into the frequency domain, preserving all information. The inverse Fourier transform converts the signal back to the time domain. The Discrete Fourier Transform (DFT) converts a discrete-time signal into its frequency domain representation. Given a sequence $x(n)$ of length N , its DFT is given by:

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N-1. \quad (1)$$

2.1 MATLAB Code for DFT

```

1  clc;
2  clear all;
3  close all;
4  a=input('Enter the sequence :');
5  N=length(a);
6  disp('The length of the sequence is:');N
7  for k=1:N
8      y(k)=0;
9      for i=1:N
10         y(k)=y(k)+a(i)*exp((-2*pi*j/N)*((i-1)*(k-1)));
11     end;
12 end;
13 k=1:N
14 disp('The result is:');y
15 figure(1);
16 subplot(211);
17 stem(k,abs(y(k)));
18 xlabel('Sample values n');
19 ylabel('Amplitude');
20 title('Magnitude response of the DFT');
21 subplot(212);
22 stem(angle(y(k))*180/pi);
23 xlabel('Sample values n');
```

```

24 ylabel('Phase');
25 title('Phase response of the DFT');

```

3 Inverse Discrete Fourier Transform (IDFT)

The IDFT reconstructs a sequence from its DFT. The inverse DFT (IDFT) reconstructs the original sequence from its frequency domain representation:

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j \frac{2\pi}{N} kn}, \quad n = 0, 1, \dots, N-1. \quad (2)$$

3.1 MATLAB Code for IDFT

```

1 clc;
2 clear all;
3 close all;
4 a=input('Enter the sequence :');
5 N=length(a);
6 disp('The length of the sequence is:');N
7 for n=1:N
8     y(n)=0;
9     for k=1:N
10        y(n)=y(n)+a(k)*exp((2*pi*j*(k-1)*(n-1))/N);
11    end;
12 end;
13 n=1:N
14 y1=1/N*y(n);
15 disp('The result is:');y1
16 figure(1);
17 stem(n,y1);
18 xlabel('Sample values n');
19 ylabel('Amplitude');
20 title('Magnitude response of the IDFT');

```

4 Finding the FFT of Different Signals

The Fast Fourier Transform (FFT) computes the DFT of a sequence efficiently. Fast Fourier transform (FFT) algorithm computes the discrete Fourier transform (DFT) of a sequence, or its inverse. Fourier analysis converts a signal from its original domain (often time or space) to a representation in the frequency domain and vice versa.

PROGRAM DESCRIPTION: In this program using the command FFT for impulse response. In the process of finding the FFT the length of the FFT is taken as N. The FFT consists of two parts: MAGNITUDE PLOT and PHASE PLOT. The magnitude plot is the absolute value of magnitude versus the samples and the phase plot is the phase angle versus the samples.

4.1 MATLAB Code for FFT

```
1 clc;
2 clear all;
3 close all;
4 t=-2:1:2;
5 y=[zeros(1,2) 1 zeros(1,2)];
6 subplot(3,1,1);
7 stem(t,y);
8 grid;
9 title('Impulse Response');
10 xlabel('Time');
11 ylabel('Amplitude');
12 N=input('Enter the length of the FFT sequence: ');
13 xk=fft(y,N);
14 magxk=abs(xk);
15 angxk=angle(xk);
16 k=0:N-1;
17 subplot(3,1,2);
18 stem(k,magxk);
19 xlabel('k');
20 ylabel('|x(k)|');
```



```
21 subplot(3,1,3);  
22 stem(k,angxk);  
23 xlabel('k');  
24 ylabel('arg(x(k))');
```

5 Lab Task

- Compute DFT of sequences by hand and using MATLAB, then plot magnitude and phase plots.
- Given sequences: $x = [1, 2, 3, 4]$, $h = [4, 3, 2, 1]$.
- Compute FFT using MATLAB for step, ramp, and exponential sequences. Plot input signals along with magnitude and phase plots.

6 Lab Assignment

Make a neat lab report.

7 Conclusion

DFT and FFT have been verified using MATLAB.

Lab 5

FIR Filter Design using MATLAB

October 16, 2025

Linear-Phase FIR Filters The shapes of impulse and frequency responses and locations of system function zeros of linear-phase FIR filters are discussed. Let: $\{h(n), \quad 0 \leq n \leq M-1\}$ be the impulse response of length M . The system function is:

$$H(z) = \sum_{n=0}^{M-1} h(n)z^{-n} \quad (1)$$

which has $M-1$ poles at the origin $z=0$ and $M-1$ zeros located anywhere in the z -plane.

$$H(z) = z^{-M+1} \sum_{n=0}^{M-1} h(n)z^{-M+1-n} \quad (2)$$

The frequency response of the function is:

$$H(e^{j\omega}) = \sum_{n=0}^{M-1} h(n)e^{-j\omega n} \quad (3)$$

1 Type-1 Linear-Phase FIR Filters

For symmetrical impulse response and odd M , In this case, $\beta=0$, $\alpha=(M-1)/2$ is an integer, and $\{h(n) = h(M-1-n), \quad 0 \leq n \leq M-1\}$ The frequency response is:

$$H(e^{j\omega}) = e^{-j\omega(M-1)/2} \sum_{n=0}^{(M-1)/2} a(n) \cos(\omega n) \quad (4)$$

where sequence $a(n)$ is obtained from $h(n)$: $\{a(0) = h(\frac{M-1}{2}), \quad a(n) = 2h(M-1-n), \quad 1 \leq n \leq \frac{M-3}{2}\}$ Since then, we have

$$H_r(\omega) = \sum_{n=0}^{(M-1)/2} a(n) \cos(\omega n) \quad (5)$$

where $H_r(\omega)$ is an amplitude response function and not a magnitude response function.

MATLAB Code for Type-1 FIR Filter

```
1 function [Hr,w,a,L]=Hr_Type1(h);
2 % Computes Amplitude response Hr(w) of a Type-1 LP FIR filter
3 % -----
```

```

4 % [Hr,w,a,L]=Hr_Type1(h)
5 % Hr = Amplitude Response
6 % w = 500 frequencies between [0,pi] over which Hr is computed
7 % a = Type-1 LP filter coefficients
8 % L = Order of Hr
9 % h = Type-1 LP filter impulse response
10
11 M = length(h);
12 L = (M-1)/2;
13 a = [h(L+1) 2*h(L:-1:1)];
14 n = [0:1:L];
15 w = [0:1:500]'*pi/500;
16 Hr = cos(w*n)*a';

```

2 Type-2 Linear-Phase FIR Filters

Symmetrical impulse response and M even. In this case $\beta=0$, $\alpha = (M-1)/2$ is not an integer and $h(n) = h(M-1-n)$, $0 \leq n \leq M-1$. Then we can show that

$$H(e^{j\omega}) = e^{-j\omega(M-1)/2} \sum_{n=1}^{M/2} b(n) \cos(\pi(n-1/2)) \quad (6)$$

where, $\{b(n) = 2h(M/2-n), \quad 1 \leq n \leq M/2\}$ Hence,

$$H_r(w) = \sum_{n=0}^{M/2} b(n) \cos(\pi(n-1/2)) \quad (7)$$

We cannot use this type i.e., symmetrical $h(n)$ and M is even for highpass or bandstop filters.

3 Type-3 Linear-Phase FIR Filters

Antisymmetric impulse response and M odd. In this case $\beta=\pi/2$, $\alpha = (M-1)/2$ is not an integer, and $h(n) = -h(M-1-n)$, $0 \leq n \leq M-1$, and $h(M-1/2) = 0$. Then

we can show that

$$H(e^{j\omega}) = e^{-j[\frac{\pi}{2} - \frac{\omega(M-1)}{2}]} \sum_{n=1}^{(M-1)/2} c(n) \sin(\omega n) \quad (8)$$

where, $\{c(n) = 2h(\frac{M-1}{2} - n), \quad 1 \leq n \leq (M-1)/2\}$ Hence,

$$H_r(w) = \sum_{n=1}^{(M-1)/2} c(n) \cos(\omega n) \quad (9)$$

Note that at $\omega = 0$ and $\omega = \pi$ we have $H_r(w) = 0$, regardless of $c(n)$ or $h(n)$. Furthermore, $e^{j\pi/2} = j$, which means that $jH_r(w)$ is purely imaginary. Hence, this type of filter is not suitable for designing a low-pass or a high-pass filter. However, this behavior is suitable for approximating ideal Hilbert transformers and differentiators.

MATLAB Code for Type-3 FIR Filter

```

1 function [Hr,w,c,L]=Hr_Type3(h);
2 % Computes Amplitude response Hr(w) of a Type-3 LP FIR filter
3 % -----
4 % [Hr,w,c,L]=Hr_Type3(h)
5 % Hr = Amplitude Response
6 % w = frequencies between [0,pi] over which Hr is computed
7 % b = Type-3 LP filter coefficients
8 % L = Order of Hr
9 % h = Type-3 LP filter impulse response
10
11 M = length(h);
12 L = (M-1)/2;
13 c = [2*h(L+1:-1:1)];
14 n = [0:1:L];
15 w = [0:1:500]'*pi/500;
16 Hr = sin(w*n)*c';

```

4 Type-4 Linear-Phase FIR Filters

Antisymmetric impulse response and M even. This case is similar to Type-2. We have

$$H(e^{j\omega}) = e^{-j[\frac{\pi}{2} - \frac{\omega(M-1)}{2}]} \sum_{n=1}^{M/2} d(n) \sin(\omega(n - 1/2)) \quad (10)$$

where, $\{d(n) = 2h(\frac{M}{2} - n), \quad 1 \leq n \leq M/2\}$ Hence,

$$H_r(w) = \sum_{n=1}^{M/2} d(n) \sin(\omega(n - 1/2)) \quad (11)$$

Note that at $\omega = 0$ and $\omega = \pi$ we have $H_r(0) = 0$ and $e^{j\pi/2} = j$, which means that $jH_r(w)$ is purely imaginary. Hence, this type of filter is suitable for approximating ideal Hilbert transformers and differentiators.

5 Window Design Techniques

Summary of commonly used window functions:

Window Name	Transition Width	Exact Values	Min. Stopband Attenuation
Rectangular	$\frac{4\pi}{M}$	$\frac{1.8\pi}{M}$	21 dB
Bartlett	$\frac{8\pi}{M}$	$\frac{6.1\pi}{M}$	25 dB
Hanning	$\frac{8\pi}{M}$	$\frac{6.2\pi}{M}$	44 dB
Hamming	$\frac{8\pi}{M}$	$\frac{6.6\pi}{M}$	53 dB
Blackman	$\frac{12\pi}{M}$	$\frac{11\pi}{M}$	74 dB

Table 1: Window Function Characteristics

MATLAB provides several routines to implement window functions discussed above.

$w = \text{boxcar}(n)$ returns the n-point rectangular window in array w.

$w = \text{triang}(n)$ returns the n-point Bartlett (Triangular) window in array w.

$w = \text{hanning}(n)$ returns the n-point symmetric Hanning window in a column vector in array w .

$w = \text{hamming}(n)$ returns the n-point symmetric Hamming window in a column vector in array w.

$w = \text{blackman}(n)$ returns the n-point symmetric Blackman window in a column vector in array w.

`w = kaiser(n,beta)` returns the beta-valued n-point Kaiser window in array `w`.

Using these routines, we can use MATLAB to design FIR filters based on the window technique, which also requires an ideal lowpass impulse response as shown below.

```

1 function hd=ideal_lp(wc,M);
2 % Ideal LowPass filter computation
3 % -----
4 % [hd] = ideal_lp(wc,M);
5 % hd = ideal impulse response between 0 to M-1
6 % wc = cutoff frequency in radians
7 % M = length of the ideal filter
8 %
9 alpha = (M-1)/2;
10 n = [0:1:(M-1)];
11 m = n - alpha +eps;          % add smallest number to avoid
    % divided by zero
12 hd = sin(wc*m)./(pi*m);

```

To display the frequency-domain plots of digital filters, MATLAB provides the `freqz` routine. Using this routine, we can develop a modified version, called `freqzm`, which returns the magnitude response in absolute scale as well as dB scale, the phase response, and the group delay response, as shown below.

```

1 function [db,mag,pha,grd,w] = freqz_m(b,a)
2 % Modified version of freqz subroutine
3 % -----
4 % [db,mag,pha,grd,w] = freqz_m(b,a)
5 % db = relative magnitude in dB computed over 0 to pi radians
6 % mag = absolute magnitude computed over 0 to pi radians
7 % pha = Phase response in radians over 0 to pi radians
8 % grd = Group delay over 0 to pi radians
9 %     w = 501 frequency samples between 0 to pi radians
10 %     b = numerator polynomial of H(z)      (for FIR: b=h)
11 %     a = denominator polynomial of H(z)    (for FIR: a=[1])
12 %
13 [H,w] = freqz(b,a,1000,'whole') ;

```

```
14 H = (H(1:1:501))';  
15 w = (w(1:1:501))';  
16 mag = abs(H);  
17 db = 20*log10((mag+eps)/max(mag));  
18 pha = angle(H);  
19 grd = grpdelay(b,a,w);
```

6 Lab Task

1. Write a MATLAB function to compute the amplitude response and the location of the zeros for Type-2 and Type-4 FIR filters.
2. Determine the type of given Linear-Phase FIR filters. Find the amplitude response and plot the impulse response, coefficients, and zero locations.
 - a) $h(n) = [-4 \ 1 \ -1 \ -2 \ 5 \ 6 \ 5 \ -2 \ -1 \ 1 \ -4]$
 - b) $h(n) = [-4 \ 1 \ -1 \ -2 \ 5 \ 6 \ 6 \ 5 \ -2 \ -1 \ 1 \ -4]$
 - c) $h(n) = [-4 \ 1 \ -1 \ -2 \ 5 \ 0 \ -5 \ 2 \ 1 \ -1 \ 4]$

Lab 6

Filter Design using FDA TOOL (FIR & IIR)

October 23, 2025

This lab will provide a basic understanding of the design and implementation of digital filters. You will design, simulate, and implement a finite impulse response (FIR) filter using MATLAB's FDA Tool.

1 Introduction

Digital Filters are among the most common DSP applications, being found in a large variety of embedded systems. This experiment involves the design, simulation and implementation of a digital filter. The filter in question is a Finite Impulse Response (FIR) filter, which presents some peculiarities. For instance, its transfer function has a numerator polynomial only (the denominator is 1), which means that there is no feedback path and therefore such filter is always stable. You will use the windowing method to design the filter, and you will have the opportunity to explore a very handy design package (the FDA Tool) to accomplish this task. You will simulate the design in Simulink and will run it in real-time on a DSP platform. After this lab, you will be able to do this:

After this lab, you will:

- Design a digital filter using FDA Tool.
- Simulate the filter in Simulink.
- Implement and test it on a DSP platform.

2 Background Reading

The digital filtering process can be described, in a very simplified manner, as the result of a convolution between the input (sampled) signal and the coefficients (also called “parameters,” or “taps”) of the digital filter. These coefficients are real-valued parameters of the polynomial which describes the filter in the z domain. By “polynomial” we mean “polynomial in the variable z^{-1} .”

If the topology of such filter has feedback, its transfer function will have polynomials for both numerator and denominator. When no feedback is present, the denominator will equal 1, and the coefficients of the numerator polynomial will represent the impulse response of the filter. This no-feedback topology describes a finite impulse response (FIR)

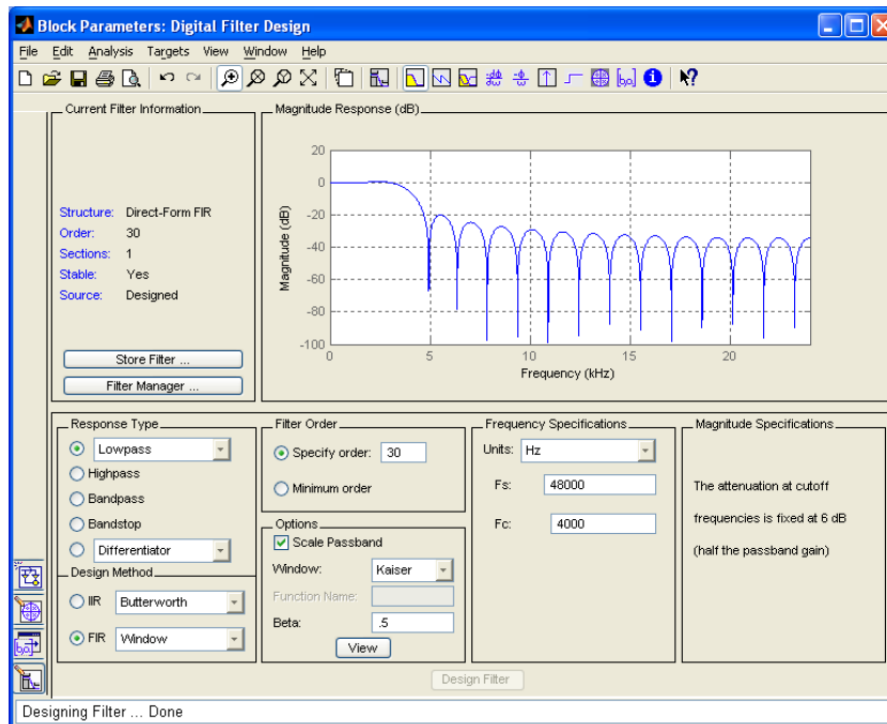


Figure 1. FDA Tool Graphic User Interface - Frequency Response Mode

filter. The one with feedback is called infinite impulse response (IIR) filter.

3 Experiment: FIR Design

This lab involves designing, simulating, and implementing an FIR filter in real-time. There are numerous software packages commercially available that will provide you with the necessary tools to design digital filters, both FIR and IIR. In this lab, you will design your filters using the Filter Design and Analysis Tool from MATLAB. The most straightforward way to start the tool is to type `fdatool` on the command line in the main MATLAB window. You can alternatively open a blank model from Simulink and drag in the FDA Tool block. When you double-click on the FDA Tool block, the GUI presented in Figure ?? will open. This block will then invoke MATLAB to generate the desired filter coefficients according to your specification.

The user interface provides you with options and spaces to set all parameters relative to the filter you want to design. In general, the most utilized display modes are the frequency response display (as in the picture above) and the impulse response display, shown in Figure ?. You should be familiar at this point with moving back and forth between the time domain and frequency domain, particularly relating to filters. On the

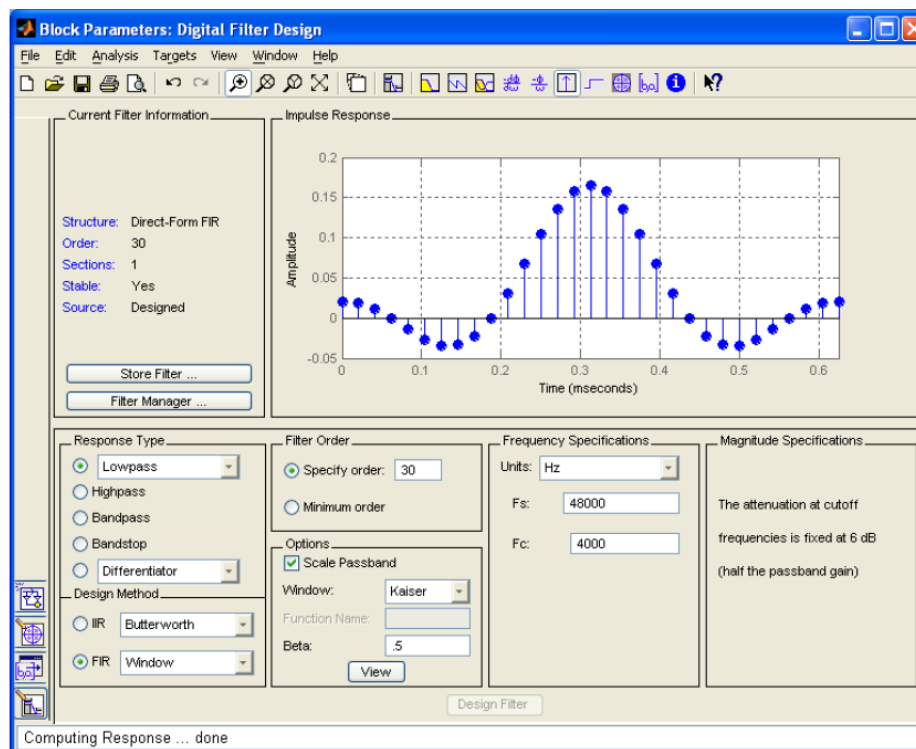


Figure 2. FDA Tool Graphic User Interface - Impulse Response Mode

FDA Toolthe impulse response plot is obtained by clicking on the upward-pointing arrow from the top menu.

A block diagram example of an FIR topology could be obtained from the FDA Toolby clicking on the “realize” button on the bottom left corner of the tool. A new window should appear with a direct form implementation of your filter , utilizing unit delay blocks, gains and adders. If you look into the gain blocks you will notice that the values there are the ones displayed in the coefficients display of your FDA Tool. The coefficients display can be obtained by clicking on the icon labeled $[b,a]$, located next to the i on the top button menu. This notation for coefficients is widely adopted, not only by MATLAB: usually b is the vector holding the numerator coefficients, whereas a is the vector holding the denominator coefficients of the filter transfer function in z . Now you will explore the FDA Toolby trying different parameters and different design methods.

The simulation parameters you will utilize in the lab samples analog signals at 48kHz i.e., F_s . In this section you will use 6kHz as your cutoff frequency, F_c . The order of your filter is 20. Also, select Low Pass as a Response Type and FIR: Window as a Design Method. The idea here is to give you a relative picture between different windows used to design lowpass FIR filters. When creating the tables, feel free to use the zoom controls

provided by the FDA Tool, or you will have to estimate the values.

4 Lab Task

4.1 Task 1

Using the window design method (guess the right type), design a LPF (Low Pass) filter with next specifications:

passband edge: 0.25,

$A_s = 40$ dB

stopband edge: 0.45,

$R_p = 1$ dB

Get the $h(n)$ of the filter and plot its amplitude response in db? And verify passband ripple R_p and stop band attenuation A_s . Hint: use the `fir1` function.

4.2 Task 2

Using frequency sampling method, Design the BPF (Band Pass) filter of next specs, using the frequency sampling method. Choose the order of the filter to be 65 so that there are two samples in the transition band. Use optimum values for these samples (see book for T1 and T2). Hint: use the `fir2` function.

lower stopband edge: 0.35

upper stopband edge: 0.65

lower passband edge: 0.45

upper passband edge: 0.55

$R_p = 1$ dB

$A_s = 55$ dB

4.3 Task 3

Design a low pass Butterworth filter to satisfy. (Hint: use Function $[b, a] = \text{afd}_{\text{butt}}(Wp, Ws, Rp, As)$).

Passband cutoff: $Pp = 0.25$

Stopband cutoff: $Ss = 0.30$

Passband ripple: $Rp = 2$ dB

Stopband ripple: $A_s = 20$ dB

A. Design the Analog LPF. Draw its frequency response.

B. Using an Impulse Invariant method, convert this analog filter to a digital one and specify its Z transform. Draw its frequency response. (T=1)

4.4 Task 4

Design a low pass digital filter using a Chebyshev-I prototype to satisfy: (Hint: use $[b, a] = \text{afd}_{chb1}(Wp, Ws, Rp, As)$).

$$Pp = 0.20,$$

$$Rp = 1dB$$

$$Ss = 0.30,$$

$$As = 30dB$$

A. Design the Analog LPF. Draw its frequency response.

B. Using a Bilinear Transformation method, convert this analog filter to a digital one and specify its Z transform. Draw its frequency response. (T=1)

Lab 7

Cepstrum Analysis and Homomorphic Deconvolution

November 13, 2025

1 Introduction

Cepstrum analysis is a vital technique in signal processing, especially in applications like speech synthesis and recognition. In this lab, you will compute the cepstrum of a speech signal, apply a liftering technique as a filtering method in cepstrum analysis, and resynthesize the signal. Accurately estimating the impulse response of the vocal tract for the targeted phoneme is crucial.

The purpose of this lab is to explore cepstrum analysis techniques using audio recordings of vowel sounds produced by both male and female speakers. The goal is to extract excitation signals from speech through homomorphic deconvolution using cepstrum domain filtering (liftering). This lab offers a comprehensive examination of cepstrum analysis and homomorphic deconvolution techniques in audio signal processing, with a focus on effectively separating the excitation and transfer function components in speech.

2 Background

2.1 Deconvolution

Deconvolution is the process of reversing the effects of convolution. In an ideal scenario, if $g(n) = f(n) * h(n)$, where $f(n)$ is the signal of interest (such as an audio signal) and its convolution with $h(n)$ represents the distortion introduced by the system—like that occurring during the recording process—it becomes essential to restore $f(n)$ from $g(n)$. This restoration can be accomplished by inverting the convolution process using $h(n)$ through various techniques, including inverse filtering.

However, when noise is present, inverse filtering can lead to significant drawbacks, particularly the amplification of noise. An alternative strategy for deconvolution involves directly separating the components $f(n)$ and $h(n)$ from $g(n)$.

2.2 Cepstrum Domain

The cepstrum is a commonly used transform that helps extract information from a person's speech signal. It allows for the separation of the excitation signal, which contains the words and pitch, from the transfer function, which reflects the quality of the voice. The

term "cepstrum" is derived from the first syllable "ceps," which is a rearrangement of the letters in the word "spectrum." This clever naming convention highlights the relationship between cepstrum and spectrum.

$$g(n) = f(n) * h(n) \quad (1)$$

In frequency domain:

$$G(\omega) = F(\omega)H(\omega) \quad (2)$$

Taking log magnitude:

$$\log |G(\omega)| = \log |F(\omega)| + \log |H(\omega)| \quad (3)$$

In the Cepstrum domain:

$$\mathcal{F}^{-1}\{\log |G(\omega)|\} = \mathcal{F}^{-1}\{\log |F(\omega)|\} + \mathcal{F}^{-1}\{\log |H(\omega)|\}$$

3 Speech Models

Within human speech, there are two methods employed to form our words. These sounds are categorized as voiced and unvoiced. For the voiced part, our throat acts like a transfer function. The vowel sounds are included in this category. The unvoiced part describes the noise-like sounds of speech. These are the sounds made with our mouth and tongue (as opposed to our throat), such as the "f" sound, the "s" sound, and the "th" sound. This way of looking at speech as two separable parts is known as the source filter model of speech or the linear separable equivalent circuit model.

4 Use of the Cepstrum in Speech Processing

By analyzing cepstrum plots, we can identify transfer functions as early sloping curves and excitation as periodic peaks in the 3–9 ms quefrency range. Liftering involves windowing in the Cepstrum domain to isolate these components.

5 Instructions

In this lab, you will gather voice samples of male and female speakers producing the five vowel sounds: “a,” “e,” “i,” “o,” “u.” You can use a built-in microphone or another recording method. Aim for ten samples in total, documenting the conditions under which they were obtained (e.g., computer, microphone type, software).

1. **Acquire Voice Samples:** Record or source at least ten voice samples from both male and female speakers. Exaggerate the vowel sounds and hold them for a longer duration. You may collaborate with others but must credit them.
2. **Compute the Cepstrum:** Compute the cepstrum for each sample and discuss differences between male and female voices, as well as among the vowels. Include figures of the time-domain and cepstrum signals. Note that female voices usually show more peaks in the cepstrum domain and explain why.
3. **Lifter the Cepstrum Signals:** Design a window (length is key) to remove the transfer function dependency, then compute the time-domain signal from the windowed result. Plot this deconvolved signal and discuss how it differs from the original recording. Your analysis is an important part of the report.

Remember, your work must be original to avoid academic consequences.

6 Deliverables

- Cover page with student details
- Executive summary of findings
- Description of signal acquisition and analysis
- Simulation plots and discussion of cepstrum differences
- Appendix with MATLAB code

Lab 8

Speech Signal Classification using Different Neural Network Algorithms

November 20, 2025

Lab Requirements

- MATLAB (preferably version 2020a or later).
- Audio Toolbox (for MFCC extraction).
- Deep Learning Toolbox (for Neural Network design).
- Two sample speech files (e.g., `speech1.wav` and `speech2.wav`).

Lab Introduction

In this lab, you will learn how to classify speech signals using two different neural network algorithms. The speech signals will be processed to extract important features such as Mel-Frequency Cepstral Coefficients (MFCCs). A Pattern Recognition Neural Network (`patternnet`) and a General Feedforward Neural Network (`feedforwardnet`) will be trained separately. Both networks will be evaluated based on accuracy, precision, recall, and F1-score. This lab demonstrates how different neural architectures affect classification performance.

Neural Network Algorithms Overview

Pattern Recognition Neural Network (`patternnet`)

The `patternnet` is specialized for classification tasks. It uses a softmax activation at the output and a single hidden layer by default. It is efficient, fast, and suitable for simple datasets and problems where high interpretability is not critical.

Feedforward Neural Network (`feedforwardnet`)

The `feedforwardnet` is a more general and flexible neural network that can have multiple hidden layers and be adapted for both regression and classification tasks. It is better suited for modeling complex patterns and non-linear boundaries in data.

Lab Tasks

1. Read two speech signals using `audioread()`.
2. Extract MFCC features from each speech signal.
3. Design and train a Pattern Recognition Neural Network (`patternnet`).
4. Design and train a Feedforward Neural Network (`feedforwardnet`).
5. Test both networks and calculate:
 - Accuracy
 - Precision
 - Recall
 - F1-Score
6. Compare the performance of both networks.

Sample MATLAB Code

```
1 clc; clear; close all;
2
3 %% Step 1: Read the speech signals
4 try
5     [x1, fs1] = audioread('speech1.wav');
6     [x2, fs2] = audioread('speech2.wav');
7 catch ME
8     error('Error reading audio files: %s', ME.message);
9 end
10
11 if fs1 ~= fs2
12     error('Sampling frequencies do not match.');
```

```
13 end
14
15 %% Step 2: Feature Extraction
```

```
16 num_cepstral_coefficients = 13; % Standard number of MFCC  
    coefficients  
17 frame_length = round(0.025 * fs1); % 25ms frame length  
18 frame_overlap = round(0.010 * fs1); % 10ms frame overlap  
19  
20 if exist('mfcc', 'file')  
21     coeff1 = mfcc(x1, fs1, 'WindowLength', frame_length,  
        'OverlapLength', frame_overlap, 'NumCoeffs',  
        num_cepstral_coefficients); % frames x features (or  
        potentially higher)  
22     coeff2 = mfcc(x2, fs2, 'WindowLength', frame_length,  
        'OverlapLength', frame_overlap, 'NumCoeffs',  
        num_cepstral_coefficients);  
23  
24     disp(['Size of coeff1 before reshaping: ',  
        num2str(size(coeff1))]);  
25     disp(['Size of coeff2 before reshaping: ',  
        num2str(size(coeff2))]);  
26  
27     % Reshape to 2D: (number of elements / number of features) x  
        number of features  
28     num_features = size(coeff1, ndims(coeff1)); % Get the last  
        dimension (likely the feature dimension)  
29     coeff1 = reshape(coeff1, [], num_features);  
30     coeff2 = reshape(coeff2, [], num_features);  
31  
32     disp(['Size of coeff1 after reshaping: ',  
        num2str(size(coeff1))]);  
33     disp(['Size of coeff2 after reshaping: ',  
        num2str(size(coeff2))]);  
34  
35 else  
36     warning('MFCC function not found. Using basic features (mean  
        and standard deviation). For better results, install the
```

```

        Audio System Toolbox.');
```

37 coeff1 = [mean(x1), std(x1)];

38 coeff2 = [mean(x2), std(x2)];

39 num_cepstral_coefficients = size(coeff1, 2); *% Update the*
 number of features

40 end

41

42 *% Take mean across frames (for each feature)*

43 feat1 = mean(coeff1, 1); *% 1 x features*

44 feat2 = mean(coeff2, 1); *% 1 x features*

45

46 *% Stack features and target variables*

47 X = [feat1; feat2]'; *% (features x samples)*

48 T = [1 0; 0 1]'; *% (classes x samples)*

49

50 *%% Step 3: Neural Network 1 - Pattern Recognition (patternnet)*

51 hiddenLayerSize1 = 10;

52 net1 = patternnet(hiddenLayerSize1);

53 net1.trainParam.showWindow = false;

54 net1 = train(net1, X, T);

55

56 YTest1 = net1(X);

57

58 *%% Step 4: Neural Network 2 - Feedforward Network*
 (feedforwardnet)

59 hiddenLayerSize2 = [15 10];

60 net2 = feedforwardnet(hiddenLayerSize2);

61 net2.trainParam.showWindow = false;

62 net2 = train(net2, X, T);

63

64 YTest2 = net2(X);

65

66 *%% Step 5: Evaluation*

67 disp('Evaluating PatternNet:');

```
68 evaluate_performance(T, YTest1);
69
70 disp('Evaluating FeedforwardNet:');
71 evaluate_performance(T, YTest2);
72
73 %% Supporting function
74 function evaluate_performance(TTrue, YPred)
75     [~, pred] = max(YPred, [], 1);
76     [~, trueclass] = max(TTrue, [], 1);
77
78     figure;
79     plotconfusion(TTrue, YPred);
80     title('Confusion Matrix');
81
82     TP = sum((pred == 1) & (trueclass == 1));
83     TN = sum((pred == 2) & (trueclass == 2));
84     FP = sum((pred == 1) & (trueclass == 2));
85     FN = sum((pred == 2) & (trueclass == 1));
86
87     accuracy = (TP + TN) / (TP + TN + FP + FN);
88     precision = TP / (TP + FP + eps);
89     recall = TP / (TP + FN + eps);
90     f1 = 2 * (precision * recall) / (precision + recall + eps);
91
92     fprintf('Accuracy: %.2f%%\n', accuracy * 100);
93     fprintf('Precision: %.2f%%\n', precision * 100);
94     fprintf('Recall: %.2f%%\n', recall * 100);
95     fprintf('F1 Score: %.2f%%\n', f1 * 100);
96 end
```

Mini Project (Extension)

Project Title:

Multi-Class Voice Command Recognition Using Neural Networks

Description: Expand the lab work by recording multiple voice commands (such as *Start*, *Stop*, *Left*, *Right*, *Up*, *Down*). Train both patternnet and feedforwardnet architectures and compare their performance. Implement a real-time system where user speech is captured and classified into different commands.

Lab 9

Speech Synthesis

November 27, 2025

Part 1: Text-to-Speech (TTS) - Grapheme-to-Phoneme (G2P) Conversion and Basic Prosodic Modeling

Objective

To understand the fundamental steps involved in converting text to a sequence of phonetic units and to introduce basic prosodic variations.

Task 1.1: Grapheme-to-Phoneme (G2P) Conversion - Rule-Based Approach

Procedure:

1. Develop a simplified rule-based G2P converter for a small subset of English words. Focus on common vowel and consonant sounds and a few basic rules (e.g., 'c' followed by 'e', 'i', or 'y' often becomes /s/, 'c' followed by other letters often becomes /k/, double letters are usually pronounced as a single sound, a few common word exceptions).
2. Represent the phonemes using a simple phonetic alphabet (e.g., ARPABET or a simplified version).
3. Implement a MATLAB function that takes an input word (string) and applies your rules to output a sequence of phonemes (cell array of strings).
4. Test your function with a set of sample words.

MATLAB Solution (Illustrative):

```
function phoneme_sequence = simple_g2p(word)
% SIMPLE_G2P A basic rule-based G2P converter for a small English subset.
%   phoneme_sequence = SIMPLE_G2P(word) converts an input word to a
%   sequence of phonemes based on a simplified set of rules.

word = lower(word); % Convert to lowercase for easier processing
phoneme_sequence = {};
```

```

i = 1;
while i <= length(word)
    if i < length(word)
        if strcmp(word(i:i+1), 'th')
            phoneme_sequence = [phoneme_sequence, {'DH'}];
            i = i + 2;
        elseif strcmp(word(i), 'c')
            if any(strcmp(word(i+1), ['e', 'i', 'y']))
                phoneme_sequence = [phoneme_sequence, {'S'}];
                i = i + 1;
            else
                phoneme_sequence = [phoneme_sequence, {'K'}];
                i = i + 1;
            end
        elseif word(i) == word(i+1) && isletter(word(i))
            % Handle double letters (simplified - assumes single sound)
            phoneme_sequence = [phoneme_sequence, {upper(word(i))}]; % Using upper
            i = i + 2;
        else
            phoneme_sequence = [phoneme_sequence, {upper(word(i))}]; % Default: u
            i = i + 1;
        end
    else
        phoneme_sequence = [phoneme_sequence, {upper(word(i))}];
        i = i + 1;
    end
end

% Handle a few exceptions
if strcmp(word, 'the')
    phoneme_sequence = {'DH', 'AH'};
end

```

```

end

% Test cases
words = {'cat', 'cent', 'apple', 'the', 'book', 'tree'};
for j = 1:length(words)
    phonemes = simple_g2p(words{j});
    disp(['Word: ', words{j}, ' -> Phonemes: ', strjoin(phonemes, '-')]);
end

```

Task 1.2: Basic Prosodic Modeling - Duration

Procedure:

1. Introduce a simple rule for assigning relative durations to phonemes based on their type (e.g., vowels have longer base duration than consonants, the last syllable in a word might have a slightly longer duration).
2. Assume a base unit of time (e.g., milliseconds).
3. Modify your MATLAB function to output not just the phoneme sequence but also a corresponding duration for each phoneme.

MATLAB Solution (Illustrative - extending the previous function):

```

function [phoneme_sequence, durations] = simple_g2p_with_duration(word)
% SIMPLE_G2P_WITH_DURATION Basic G2P with simple duration modeling.
% [phoneme_sequence, durations] = SIMPLE_G2P_WITH_DURATION(word) converts
% a word to phonemes and assigns basic durations.

word = lower(word);
phoneme_sequence = {};
durations = [];
vowels = 'aeiou';
base_vowel_duration = 100; % ms
base_consonant_duration = 50; % ms
final_syllable_duration_increase = 20; % ms (very simplistic)

```

```
% (G2P rules from the previous function remain the same - omitted for brevity)
% ...

i = 1;
while i <= length(word)
    % (G2P logic here - similar to before)
    phoneme = ''; % Store the extracted phoneme

    if i < length(word)
        if strcmp(word(i:i+1), 'th')
            phoneme = 'DH';
            i = i + 2;
        elseif strcmp(word(i), 'c')
            if any(strcmp(word(i+1), vowels))
                phoneme = 'S';
                i = i + 1;
            else
                phoneme = 'K';
                i = i + 1;
            end
        elseif word(i) == word(i+1) && isletter(word(i))
            phoneme = upper(word(i));
            i = i + 2;
        else
            phoneme = upper(word(i));
            i = i + 1;
        end
    else
        phoneme = upper(word(i));
        i = i + 1;
    end
end
```

```

    phoneme_sequence = [phoneme_sequence, {phoneme}];

    % Assign duration based on phoneme type (very basic)
    if any(strcmp(lower(phoneme), regexp(vowels, '.', 'match'))))
        duration = base_vowel_duration;
    else
        duration = base_consonant_duration;
    end
    durations = [durations, duration];
end

% Very basic final syllable lengthening (highly simplified)
if ~isempty(durations)
    durations(end) = durations(end) + final_syllable_duration_increase;
end

% Handle 'the' exception with duration
if strcmp(word, 'the')
    phoneme_sequence = {'DH', 'AH'};
    durations = [base_consonant_duration, base_vowel_duration + final_syllable_du
end

end

% Test cases
words = {'cat', 'apple', 'hello'};
for j = 1:length(words)
    [phonemes, durs] = simple_g2p_with_duration(words{j});
    disp(['Word: ', words{j}, ' -> Phonemes: ', strjoin(phonemes, '-'), ' -> Duration
end

```

Lab Report Questions (for Part 1)

1. Explain the process of Grapheme-to-Phoneme (G2P) conversion. Why is it a crucial step in TTS?
2. Discuss the challenges of rule-based G2P conversion. What are some limitations of the rules you implemented?
3. How can more sophisticated techniques (e.g., statistical models, neural networks) improve G2P conversion?
4. Explain the concept of prosody in speech. Why is modeling prosody important for natural-sounding speech?
5. Describe the simple duration model you implemented. What are some other prosodic features that could be modeled?

Lab 10

OPEN ENDED LAB

December 4, 2025

Group A – Advanced Speech Enhancement with Cepstrum-Based VAD

Objective

Design a MATLAB-based speech enhancement system with noise suppression, voice activity detection (VAD), cepstrum-based pitch tracking, and quality analysis.

Tasks

1. Record a noisy real-world speech sample.
2. Use short-time energy and cepstrum peak detection for VAD.
3. Estimate noise during unvoiced segments and perform spectral subtraction.
4. Apply FIR bandpass filtering before and after enhancement.
5. Use real cepstrum to estimate pitch contour over time.
6. Evaluate signal using SNR and spectrogram plots.

Open-Ended Lab Task – Group A

Title: Designing a Real-Time Voice Enhancement System

Objective

To develop and analyze a MATLAB-based system that enhances voice quality by reducing noise and emphasizing speech characteristics using DSP techniques.

Tasks

1. Record a noisy speech sample.
2. Apply DFT to analyze spectral content.
3. Design and apply FIR/IIR filter for noise suppression.

4. Use real cepstrum to estimate pitch and clean signal.
5. Visualize and compare before/after effects.

Group B – Speaker Classification Using MFCC and Cepstrum

Objective

Build a machine learning model to classify speakers using MFCCs and cepstrum features.
Test robustness to noise and filtering.

Tasks

1. Collect at least 3 speakers' recordings with 5 utterances each.
2. Extract MFCC and cepstrum features for each utterance.
3. Apply preprocessing: filtering and noise addition.
4. Train KNN and SVM classifiers and compare results.
5. Analyze classifier performance on clean vs noisy signals.

Open-Ended Lab Task – Group B

Title: Speaker Classification System Using Cepstral Features

Objective

To build a basic speaker classification model using cepstral coefficients and classify between two speakers.

Tasks

1. Record two speaker samples.
2. Preprocess using filters and convolution.
3. Extract real cepstrum / MFCCs.
4. Classify using Euclidean distance.

5. Analyze accuracy.

Lab 11

Mel-Frequency Cepstral Coefficients (MFCCs) for Speech Recognition

December 11, 2025

Objective

To understand and implement the steps involved in extracting Mel-Frequency Cepstral Coefficients (MFCCs) from a speech signal using MATLAB. This lab will provide a detailed walkthrough of each stage of the MFCC extraction process and culminate in a small speech recognition-related project.

Introduction to MFCCs

Mel-Frequency Cepstral Coefficients (MFCCs) are a widely used feature extraction technique in speech recognition. They aim to mimic the human auditory system's perception of sound frequencies. By transforming the raw audio signal into a set of coefficients that represent the spectral envelope, MFCCs capture the essential characteristics of speech relevant for tasks like speaker identification and speech recognition.

The key steps involved in MFCC extraction are:

1. Framing
2. Windowing
3. Fast Fourier Transform (FFT)
4. Mel Filter Bank Application
5. Logarithm of Filter Bank Energies
6. Discrete Cosine Transform (DCT)

Materials

- MATLAB software installed.
- A recorded speech signal (preferably a `.wav` file).

Procedure

Step 1: Framing

Speech signals are non-stationary, meaning their properties change over time. To analyze these changes, the signal is divided into short-duration frames. Typical frame lengths range from 20ms to 40ms with an overlap between consecutive frames (e.g., 50

MATLAB Code Snippet:

```
% Load the speech signal
[speechSignal, sampleRate] = audioread('your_speech_file.wav');

% Define frame parameters
frameDuration = 0.025; % 25ms frame duration
frameOverlap = 0.010; % 10ms frame overlap

% Calculate frame length and step in samples
frameLengthSamples = round(frameDuration * sampleRate);
frameStepSamples = round(frameOverlap * sampleRate);

% Perform framing
numFrames = floor((length(speechSignal) - frameLengthSamples) / frameStepSamples) + 1;
frames = zeros(frameLengthSamples, numFrames);

for i = 1:numFrames
    startIndex = (i - 1) * frameStepSamples + 1;
    endIndex = startIndex + frameLengthSamples - 1;
    if endIndex <= length(speechSignal)
        frames(:, i) = speechSignal(startIndex:endIndex);
    else
        % Pad the last frame with zeros if necessary
        padding = endIndex - length(speechSignal);
        frames(:, i) = [speechSignal(startIndex:endIndex-padding); zeros(padding, 1)]
```



```
        end
    end

    disp(['Number of frames: ', num2str(numFrames)]);
    disp(['Size of each frame: ', num2str(size(frames(:,1)))]);
```

Explanation: This code first loads the speech signal and defines the desired frame duration and overlap. It then calculates the frame length and step in terms of the number of samples based on the sampling rate. Finally, it iterates through the speech signal, extracting frames of the specified length with the defined overlap and stores them in the `frames` matrix.

Step 2: Windowing

To minimize the spectral leakage at the boundaries of each frame caused by the abrupt truncation, a window function is applied to each frame. The Hamming window is a commonly used window function for this purpose.

MATLAB Code Snippet:

```
% Apply Hamming window to each frame
window = hamming(frameLengthSamples);
windowedFrames = frames .* repmat(window, 1, numFrames);

disp(['Size of windowed frames: ', num2str(size(windowedFrames))]);
```

Explanation: This code generates a Hamming window of the same length as each frame. Then, it multiplies each frame in the `frames` matrix element-wise with the Hamming window using the `repmat` function to apply the same window to all frames.

Step 3: Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is applied to each windowed frame to convert it from the time domain to the frequency domain. The magnitude spectrum obtained from the FFT represents the distribution of energy across different frequencies in the frame.

MATLAB Code Snippet:

```
% Perform FFT on each windowed frame
NFFT = 512; % Typically a power of 2, often greater than frame length
magnitudeSpectrum = abs(fft(windowedFrames, NFFT));

disp(['Size of magnitude spectrum: ', num2str(size(magnitudeSpectrum))]);
```

Explanation: Here, we define the number of points for the FFT (NFFT), which is often chosen to be a power of 2 and greater than or equal to the frame length to provide a denser frequency spectrum. The `fft` function computes the discrete Fourier transform, and then the absolute value (`abs`) is taken to obtain the magnitude spectrum.

Step 4: Mel Filter Bank Application

The magnitude spectrum is then passed through a bank of triangular filters spaced linearly at low frequencies and logarithmically at high frequencies. This filter bank mimics the non-linear frequency resolution of the human ear. The number of filters is typically around 20-40.

MATLAB Code Snippet:

```
% Define Mel filter bank parameters
numFilters = 26;
lowFreq = 0;
highFreq = sampleRate / 2;

% Convert Hz to Mel
melLow = 2595 * log10(1 + lowFreq / 700);
melHigh = 2595 * log10(1 + highFreq / 700);

% Generate Mel-spaced points
melPoints = linspace(melLow, melHigh, numFilters + 2);

% Convert Mel to Hz
hzPoints = 700 * (10.^(melPoints / 2595) - 1);
```

```
% Create triangular filters
filterBank = zeros(numFilters, NFFT / 2 + 1);
for i = 1:numFilters
    leftFreq = hzPoints(i);
    centerFreq = hzPoints(i + 1);
    rightFreq = hzPoints(i + 2);

    for j = 1:(NFFT / 2 + 1)
        freq = (j - 1) * sampleRate / NFFT;
        if (freq >= leftFreq) && (freq <= centerFreq)
            filterBank(i, j) = (freq - leftFreq) / (centerFreq - leftFreq);
        elseif (freq > centerFreq) && (freq <= rightFreq)
            filterBank(i, j) = (rightFreq - freq) / (rightFreq - centerFreq);
        end
    end
end

% Apply filter bank to the magnitude spectrum
filterBankEnergies = filterBank * magnitudeSpectrum(1:NFFT/2+1, :);

disp(['Size of filter bank energies: ', num2str(size(filterBankEnergies))]);

% Optional: Visualize the filter bank
figure;
plot(hzPoints);
title('Mel Filter Bank Frequencies');
xlabel('Filter Number');
ylabel('Frequency (Hz)');

figure;
imagesc(filterBank);
title('Mel Filter Bank');
```

```
xlabel('FFT Bins');  
ylabel('Filter Number');  
colorbar;
```

Explanation: This section first defines the parameters for the Mel filter bank, including the number of filters and the frequency range. It then converts the frequency range to the Mel scale and generates equally spaced points in the Mel domain. These Mel points are then converted back to the Hertz scale to define the center frequencies of the triangular filters. Finally, it creates the triangular filters and applies them to the magnitude spectrum of each frame by matrix multiplication. The result `filterBankEnergies` contains the energy in each filter bank for each frame.

Step 5: Logarithm of Filter Bank Energies

The energies in each filter bank are compressed using a logarithm. This step is motivated by the logarithmic sensitivity of human hearing to sound intensity.

MATLAB Code Snippet:

```
% Take the logarithm of the filter bank energies  
logFilterBankEnergies = log(filterBankEnergies);  
  
disp(['Size of log filter bank energies: ', num2str(size(logFilterBankEnergies))]);
```

Explanation: This code simply applies the natural logarithm to each of the filter bank energies obtained in the previous step.

Step 6: Discrete Cosine Transform (DCT)

The Discrete Cosine Transform (DCT) is applied to the logarithm of the filter bank energies. The DCT decorrelates the filter bank energies and concentrates the information in the lower-order coefficients. The first few coefficients (typically 12-13) are retained as the MFCCs.

MATLAB Code Snippet:

```
% Apply Discrete Cosine Transform (DCT)
```

```
numCepstralCoeff = 13;
mfccs = dct(logFilterBankEnergies);
mfccs = mfccs(1:numCepstralCoeff, :);

disp(['Size of MFCCs: ', num2str(size(mfccs))]);

% Optional: Visualize the MFCCs
figure;
imagesc(mfccs);
title('Mel-Frequency Cepstral Coefficients (MFCCs)');
xlabel('Frame Number');
ylabel('Cepstral Coefficient');
colorbar;
```

Explanation: The `dct` function in MATLAB computes the Discrete Cosine Transform of the log filter bank energies. We then typically keep only the first few coefficients (e.g., 13) as these contain the most important information about the spectral envelope.

Task 1: Visualizing Speech Features

Objective: To visualize the extracted MFCC features for different speech sounds or words.

Steps:

1. Record or obtain short audio samples of different speech sounds (e.g., vowels like 'a', 'e', 'i', 'o', 'u') or simple words (e.g., 'yes', 'no', 'hello').
2. For each audio sample, perform the MFCC extraction process as described in the lab.
3. Visualize the resulting MFCC matrices as images (as shown in the optional visualization in Step 6). Each row of the image represents a cepstral coefficient, and each column represents a frame in the speech signal. The color intensity can represent the magnitude of the coefficient.

4. Observe and compare the patterns in the MFCC images for different speech sounds or words. Discuss any noticeable differences or similarities.

Expected Outcome: You should be able to generate MFCC feature matrices for different speech segments and visualize them. By comparing these visualizations, you can gain an intuitive understanding of how the spectral characteristics of different sounds are represented by the MFCC features.

Task 2: Impact of Frame Size and Overlap on MFCCs

Objective: To investigate how different frame sizes and overlap percentages affect the extracted MFCC features.

Procedure:

1. Load a speech signal.
2. Implement the MFCC extraction process while varying the following parameters:
 - Frame duration: Experiment with values like 15ms, 25ms (default), and 40ms.
 - Frame overlap: Try different overlap percentages such as 0
3. For each combination of frame size and overlap, visualize the resulting MFCC matrix as an image.
4. Analyze the visualizations. How do changes in frame size and overlap affect the temporal resolution (horizontal axis) and the overall appearance of the MFCC features?
5. Discuss the trade-offs between temporal resolution and spectral smoothing when choosing frame size and overlap.

MATLAB Hints: You can create a loop to iterate through different values of `frameDuration` and `frameOverlap` and rerun the MFCC extraction code for each combination. Store or display the resulting MFCC matrices for comparison.

Task 3: Effect of the Number of Mel Filters and Cepstral Coefficients

Objective: To understand the influence of the number of Mel filters and the number of retained cepstral coefficients on the MFCC representation.

Procedure:

1. Load a speech signal.
2. Implement the MFCC extraction process, but this time vary:
 - Number of Mel filters (`numFilters`): Experiment with values like 20, 26 (default), and 40.
 - Number of cepstral coefficients (`numCepstralCoeff`): Try retaining 10, 13 (default), and 20 coefficients.
3. For each combination, visualize the resulting MFCC matrix.
4. Observe and describe how the number of Mel filters affects the frequency resolution of the filter bank and subsequently the MFCCs.
5. Analyze how the number of cepstral coefficients retained impacts the amount of detail captured in the spectral envelope. What information might be lost or retained by changing this number?

MATLAB Hints: Modify the values of `numFilters` and `numCepstralCoeff` in your MFCC extraction code and observe the changes in the output `mfccs` matrix and its visualization.

Task 4: Basic Speaker Identification using MFCCs (Mini-Project Extension)

Objective: To build a very basic speaker identification system using MFCC features and a simple classifier.

Procedure:

1. Record short speech samples (e.g., saying a specific word or phrase) from a small number of different speakers (e.g., 2-3 speakers, record 5-10 samples per speaker).
2. For each speech sample, extract the MFCC features (you can use the code developed in the earlier tasks).
3. For each speaker, calculate the average MFCC vector across all their speech samples. This will serve as a simple "model" for each speaker.
4. Take a new, unseen speech sample from one of the speakers. Extract its MFCC features and calculate the average MFCC vector for this sample.
5. Compare the average MFCC vector of the unseen sample to the average MFCC vectors of each speaker (the models) using a distance metric like Euclidean distance.
6. Identify the speaker whose model has the minimum distance to the unseen sample's average MFCC vector.
7. Evaluate the performance of this basic system with a few test samples. Discuss the limitations of this simple approach and potential improvements.

MATLAB Hints: You can store the MFCC features for each speaker in separate matrices or cell arrays. Use the `mean` function to calculate the average MFCC vectors. The `pdist` function in MATLAB can be used to calculate the Euclidean distance between vectors.

Lab 12

Speech Recognition using Mel-Frequency Cepstral Coefficients (MFCCs)

December 16, 2025

1 Introduction

This lab outlines a series of tasks designed to introduce students to the fundamental concepts and practical implementation of speech recognition using Mel-Frequency Cepstral Coefficients (MFCCs). Speech recognition systems aim to enable computers to understand spoken language, a task crucial for various applications ranging from voice assistants to hands-free control systems. MFCCs are widely used features in speech processing due to their ability to effectively represent the spectral envelope of speech, which is important for distinguishing different phonemes.

The lab exercises will cover:

- Extraction and visualization of MFCC features from speech signals.
- Preparation and normalization of these features for machine learning algorithms.
- Implementation of simple classification models (e.g., K-Nearest Neighbors) for isolated word recognition.
- An introduction to sequence-based comparison using Dynamic Time Warping (DTW).
- A conceptual overview of more advanced techniques like Hidden Markov Models (HMMs) and Deep Learning in speech recognition.

1.1 Objective

To implement a basic speech recognition system capable of identifying spoken words or phonemes using Mel-Frequency Cepstral Coefficients (MFCCs) as features.

1.2 Prerequisites

- Understanding of digital signal processing fundamentals (sampling, FFT, filters).
- Familiarity with MFCC feature extraction (you should have already performed this).
- Basic knowledge of classification algorithms (e.g., K-Nearest Neighbors, Support Vector Machines).

- Programming environment with MATLAB and relevant toolboxes (e.g., Audio Toolbox, Statistics and Machine Learning Toolbox).
- A small, labeled speech dataset (e.g., isolated digits, a few command words like "yes", "no", "up", "down"). Create an 'audio/' folder in your MATLAB directory with these files.

Lab Task 1: MFCC Feature Preparation and Visualization

Goal

To ensure MFCC features are correctly extracted and prepared for classification, and to understand their characteristics through visualization and normalization.

Procedure

[label=0.] **Review MFCC Extraction:**

1.
 - Load a short speech audio file (e.g., a single spoken digit or command word).
 - Use MATLAB's 'audioFeatureExtractor' or similar functions to re-extract MFCC features. Pay attention to parameters like 'NumCoefficients', window size, and overlap.
 - Visualize the raw speech waveform using 'plot'.
 - Visualize the extracted MFCCs as a 2D matrix (time vs. MFCC coefficients) using 'imagesc'. Label your axes clearly.

2. Feature Normalization:

- Understand why feature normalization (e.g., mean normalization, standardization) is important for many machine learning algorithms.
- Implement a method to normalize your MFCC features. This typically involves subtracting the mean and dividing by the standard deviation for each MFCC coefficient across all frames of an utterance.

- Apply this normalization to the extracted MFCCs and visualize the normalized features.

3. Feature Aggregation (Optional but Recommended for Simplicity):

- For initial isolated word recognition tasks, it's common to represent each word by a single feature vector. A simple way to achieve this is by averaging the MFCC frames over the entire duration of the word.
- Implement this simple aggregation technique (e.g., using 'mean' along the time dimension) to convert a time-series of MFCCs for a single word into a single feature vector.

MATLAB Code for Lab Task 1

Place the following MATLAB code into a script named '*labtask1.m*'. Remember to adjust 'audioFolderPath' and 'sampleAudioFile' to match your local setup.

```
1 % lab_task_1.m
2
3 clear; clc; close all;
4
5 % --- Configuration ---
6 audioFolderPath = 'audio/'; % Adjust to your audio files
   directory
7 sampleAudioFile = 'yes_1.wav'; % A sample file to test MFCC
   extraction
8 n_mfcc_coeffs = 13; % Number of MFCC coefficients
9
10 % --- 1. Review MFCC Extraction ---
11 fprintf('--- Lab Task 1: MFCC Feature Preparation and
   Visualization ---\n');
12
13 % Load a sample audio file
14 [y, fs] = audioread(fullfile(audioFolderPath, sampleAudioFile));
15
16 % Ensure mono and normalize (optional but good practice)
```

```

17 if size(y, 2) > 1
18     y = mean(y, 2); % Convert to mono
19 end
20 y = y / max(abs(y)); % Normalize amplitude
21
22 fprintf('Loaded audio: %s (Sample Rate: %d Hz)\n',
        sampleAudioFile, fs);
23
24 % Extract MFCCs (using audioFeatureExtractor)
25 afe = audioFeatureExtractor( ...
26     'SampleRate', fs, ...
27     'Window', hann(round(0.025*fs), 'periodic'), ... % 25 ms
        window
28     'OverlapLength', round(0.010*fs), ...           % 10 ms
        overlap
29     'mfcc', true, ...
30     'NumBands', 26, ...                             % Number of
        filter bank bands (common)
31     'NumCoefficients', n_mfcc_coeffs, ...           % Number of
        MFCC coefficients
32     'Delta', false, ...                             % Don't
        compute delta features yet
33     'DeltaDelta', false);                           % Don't
        compute delta-delta features yet
34
35 features = extract(afe, y); % This will be [numFrames x
        numFeatures]
36
37 fprintf('MFCCs extracted. Dimensions: %d frames x %d
        coefficients\n', size(features, 1), size(features, 2));
38
39 % Visualize speech waveform
40 figure;
41 subplot(2,1,1);

```

```

42 plot((0:length(y)-1)/fs, y);
43 title(['Speech Waveform: ' sampleAudioFile]);
44 xlabel('Time (s)');
45 ylabel('Amplitude');
46 grid on;
47
48 % Visualize MFCCs
49 subplot(2,1,2);
50 imagesc(features'); % Transpose for imagesc: rows are coeffs,
    columns are frames
51 colorbar;
52 axis xy; % Origin at bottom left
53 title('MFCC Features');
54 xlabel('Frame Index');
55 ylabel('MFCC Coefficient Index');
56 colormap(jet); % Or another suitable colormap
57 set(gca, 'YTick', 1:size(features,2)); % Show individual
    coefficient indices
58 grid on;
59
60 % --- 2. Feature Normalization ---
61 % Global mean-variance normalization (per coefficient)
62 mean_features = mean(features, 1); % Mean of each column
    (coefficient)
63 std_features = std(features, 0, 1); % Standard deviation of each
    column
64 normalized_features = (features - mean_features) ./
    (std_features + eps); % Add eps to prevent division by zero
65
66 fprintf('\nMFCC features normalized (mean 0, std 1).\n');
67
68 % Visualize normalized MFCCs
69 figure;
70 imagesc(normalized_features'); % Transpose for imagesc

```

```

71 colorbar;
72 axis xy;
73 title('Normalized MFCC Features');
74 xlabel('Frame Index');
75 ylabel('MFCC Coefficient Index');
76 colormap(jet);
77 set(gca, 'YTick', 1:size(normalized_features,2));
78 grid on;
79
80 % --- 3. Feature Aggregation (Simple Mean) ---
81 % For isolated word recognition, represent entire word by a
    single vector
82 aggregated_feature = mean(normalized_features, 1); % Mean of all
    frames for this word
83 fprintf('Aggregated feature vector (mean of frames). Dimensions:
    1 x %d\n', length(aggregated_feature));
84
85 % Example of what an aggregated feature vector looks like
86 disp('Aggregated Feature Vector for sample audio:');
87 disp(aggregated_feature);
88
89 fprintf('\nLab Task 1 Complete. Review plots and console
    output.\n');

```

Listing 1: MATLAB Code for Lab Task 1

Lab Task 2: Building a Simple Classifier (Isolated Word Recognition)

Goal

To implement a basic classification model to recognize isolated words using aggregated MFCC features.

Procedure

[label=0.]Dataset Preparation:

1.
 - Choose a small dataset of isolated words (e.g., "yes", "no", "up", "down", "left", "right" spoken by a few individuals). Ensure your 'audio/' folder contains these.
 - For each audio file in your dataset:
 - Extract MFCC features.
 - Apply feature normalization (as performed in Lab Task 1).
 - Aggregate the MFCC frames into a single feature vector per word (e.g., mean of all frames).
 - Organize your data into a feature matrix ('allFeatures' or 'X') where each row is an aggregated feature vector for a word, and a label vector ('allLabels' or 'y').

2. Train-Test Split:

- Divide your dataset into training and testing sets (e.g., 70% train, 30% test) using 'cvpartition'. Explain the importance of this step in evaluating machine learning models.

3. K-Nearest Neighbors (KNN) Classifier:

- Implement a KNN classifier using MATLAB's 'fitcknn' function.
- Train the KNN model on your training data.
- Evaluate the model's performance on the test data using metrics like accuracy (percentage of correctly classified samples).
- Experiment with different values of 'k' (number of neighbors) and observe the effect on accuracy.

4. Support Vector Machine (SVM) Classifier (Optional):

- Implement an SVM classifier using MATLAB's 'fitcsvm'.

- Train and evaluate the SVM model.
- Compare its performance with KNN.

MATLAB Code for Lab Task 2

Place the following MATLAB code into a script named ‘*labtask2.m*’. Ensure your ‘audio/’ folder contains the words specified in ‘wordList’.

```

1 % lab_task_2.m
2 clear; clc; close all;
3 % --- Configuration ---
4 audioFolderPath = 'audio/'; % Adjust to your audio files
   directory
5 wordList = {'yes', 'no', 'up', 'down'}; % List of words you
   expect in your audio folder
6 n_mfcc_coeffs = 13; % Number of MFCC coefficients
7 % --- 1. Dataset Preparation ---
8 fprintf('--- Lab Task 2: Building a Simple Classifier (Isolated
   Word Recognition) ---\n');
9 allFeatures = [];
10 numericLabels = []; % Numeric labels for classification
11
12 % To map string labels to numbers
13 labelMapping = containers.Map;
14 currentLabelIdx = 0;
15
16 fprintf('Loading and processing audio files...\n');
17 for i = 1:length(wordList)
18     currentWord = wordList{i};
19     % Assign a unique numeric label to each word
20     if ~labelMapping.isKey(currentWord)
21         currentLabelIdx = currentLabelIdx + 1;
22         labelMapping(currentWord) = currentLabelIdx;
23     end
24

```

```

25     % Search for files for the current word (e.g.,
        'yes_1.wav', 'yes_2.wav')
26     audioFiles = dir(fullfile(audioFolderPath, [currentWord
        '*.wav']));
27
28     for j = 1:length(audioFiles)
29         fileName = audioFiles(j).name;
30         filePath = fullfile(audioFolderPath, fileName);
31
32         [y, fs] = audioread(filePath);
33         if size(y, 2) > 1
34             y = mean(y, 2); % Convert to mono
35         end
36         y = y / max(abs(y)); % Normalize amplitude
37
38         % Extract MFCCs
39         afe = audioFeatureExtractor( ...
40             'SampleRate', fs, ...
41             'Window', hann(round(0.025*fs), 'periodic'),
42             ...
43             'OverlapLength', round(0.010*fs), ...
44             'mfcc', true, 'NumCoefficients',
45             n_mfcc_coeffs, ...
46             'Delta', false, 'DeltaDelta', false);
47         features = extract(afe, y);
48
49         % Normalize features (per utterance)
50         % For simplicity, we normalize per utterance here. A
51         global normalization
52         % across the entire dataset (training set only) is
53         also common.
54         features = (features - mean(features, 1)) ./
55         (std(features, 0, 1) + eps);

```

```

52         % Aggregate features (mean over time)
53         aggregated_feature = mean(features, 1);
54
55         allFeatures = [allFeatures; aggregated_feature];
56         % #ok<AGROW>
57         numericLabels = [numericLabels;
58         labelMapping(currentWord)]; % #ok<AGROW>
59     end
60 end
61 fprintf('Finished loading %d samples.\n', size(allFeatures, 1));
62 fprintf('Label mapping: \n');
63 disp(labelMapping.keys);
64 disp(labelMapping.values);
65
66 % --- 2. Train-Test Split ---
67 rng(1); % For reproducibility of the split
68 cv = cvpartition(numericLabels, 'HoldOut', 0.3); % 30% for
69         testing
70
71 idxTrain = training(cv);
72 idxTest = test(cv);
73
74 XTrain = allFeatures(idxTrain, :);
75 yTrain = numericLabels(idxTrain);
76 XTest = allFeatures(idxTest, :);
77 yTest = numericLabels(idxTest);
78
79 fprintf('Dataset split: %d training samples, %d testing
80         samples.\n', size(XTrain, 1), size(XTest, 1));
81
82 % --- 3. K-Nearest Neighbors (KNN) Classifier ---
83 fprintf('\nTraining K-Nearest Neighbors (KNN) Classifier...\n');
84
85 k_values = [1, 3, 5]; % Experiment with different K values

```

```
82 bestAccuracy = 0;
83 bestK = 0;
84
85 for k = k_values
86     % Create and train KNN model
87     mdl = fitcknn(XTrain, yTrain, 'NumNeighbors', k,
88         'Standardize', false); % Data already normalized
89
90     % Predict on test data
91     yPred = predict(mdl, XTest);
92
93     % Evaluate accuracy
94     accuracy = sum(yPred == yTest) / length(yTest);
95     fprintf('    K = %d, Test Accuracy: %.2f%%\n', k, accuracy
96         * 100);
97
98     if accuracy > bestAccuracy
99         bestAccuracy = accuracy;
100         bestK = k;
101     end
102 end
103
104 fprintf('Best KNN accuracy: %.2f%% with K = %d\n', bestAccuracy
105     * 100, bestK);
106
107 % --- 4. Support Vector Machine (SVM) Classifier (Optional) ---
108 fprintf('\nTraining Support Vector Machine (SVM) Classifier
109     (Optional)...\n');
110
111 % Create and train SVM model (linear kernel as a start)
112 % You can experiment with different kernels ('linear', 'rbf',
113     'polynomial')
```

```
109 % and C parameter (regularization).
110 svm_mdl = fitcsvm(XTrain, yTrain, 'KernelFunction', 'linear',
    'Standardize', false);
111
112 % Predict on test data
113 yPred_svm = predict(svm_mdl, XTest);
114
115 % Evaluate accuracy
116 accuracy_svm = sum(yPred_svm == yTest) / length(yTest);
117 fprintf('    SVM (Linear Kernel) Test Accuracy: %.2f%%\n',
    accuracy_svm * 100);
118
119 fprintf('\nLab Task 2 Complete. Review console output for
    accuracies.\n');
```

Listing 2: MATLAB Code for Lab Task 2

Questions for Lab Task 2

- What are the advantages and disadvantages of using KNN for this task? How does the choice of ‘k’ influence the decision boundary?
- How do SVMs fundamentally differ from KNN? When might you prefer one over the other for speech recognition?
- What is the purpose of the train-test split, and why is it crucial for reliable model evaluation?

Lab 13

Speech Recognition using Mel-Frequency Cepstral Coefficients (MFCCs)-Part B

December 18, 2025

Lab Task 1: Introduction to Sequence-Based Classification (Dynamic Time Warping - DTW)

Goal

To understand how to compare variable-length MFCC sequences directly without aggregation, leading to more robust recognition for variable-length speech signals.

Procedure

[label=0.] **Understanding DTW:**

1.
 - Research and explain the concept of Dynamic Time Warping (DTW) in the context of sequence alignment.
 - Discuss why DTW is particularly useful for comparing speech signals, which can vary significantly in speaking rate, duration, and pronunciation.

2. DTW Implementation for Word Comparison:

- Select two recordings of the *same* word spoken differently (e.g., one fast, one slow) and one recording of a *different* word.
- Extract their MFCC sequences (no aggregation this time). Ensure these are normalized per utterance.
- Implement a simple DTW function or utilize a pre-existing one (if available in your MATLAB environment or from the File Exchange) to calculate the DTW distance between the two MFCC sequences. A basic ‘*dtw_distance*’ function is provided in the code section.
- Compare the DTW distance between the two "same word" utterances and between a "same word" and a "different word" utterance.

3. Basic DTW-based Classifier (Conceptual/Small Scale):

- For a very small dataset of isolated words (e.g., 3-5 words, 2-3 examples per word for training), conceptualize a DTW-based classifier.

- When given a test utterance, calculate its DTW distance to **every** training utterance.
- Classify the test utterance as the label of the training utterance with the minimum DTW distance.
- Discuss the accuracy and computational challenges of this approach.

MATLAB Code for Lab Task 3

Place the following MATLAB code into a script named *'labtask3.m'*. Note that a simple *'dtw_{distance}'* function is provided for demonstration. For robustness, you might consider a more optimized or library-based DTW implementation for larger datasets.

```
1 % lab_task_3.m
2
3 clear; clc; close all;
4
5 % --- Configuration ---
6 audioFolderPath = 'audio/'; % Adjust to your audio files
   directory
7 n_mfcc_coeffs = 13;
8
9 fprintf('--- Lab Task 3: Introduction to Sequence-Based
   Classification (DTW) ---\n');
10
11 % --- 2. DTW Implementation for Word Comparison ---
12 % Example: Select two recordings of the same word (e.g., 'yes')
13 file1 = 'yes_1.wav';
14 file2 = 'yes_2.wav'; % Another recording of 'yes' (ideally
   different tempo)
15
16 [y1, fs1] = audioread(fullfile(audioFolderPath, file1));
17 [y2, fs2] = audioread(fullfile(audioFolderPath, file2));
18
19 % Ensure mono and normalize
```



```

20 if size(y1, 2) > 1; y1 = mean(y1, 2); end
21 if size(y2, 2) > 1; y2 = mean(y2, 2); end
22 y1 = y1 / max(abs(y1));
23 y2 = y2 / max(abs(y2));
24
25 % Extract MFCC sequences (no aggregation this time)
26 afe = audioFeatureExtractor( ...
27     'SampleRate', fs1, ... % Assuming same sample rate
28     'Window', hann(round(0.025*fs1), 'periodic'), ...
29     'OverlapLength', round(0.010*fs1), ...
30     'mfcc', true, 'NumCoefficients', n_mfcc_coeffs, ...
31     'Delta', false, 'DeltaDelta', false);
32
33 mfccs1 = extract(afe, y1);
34 mfccs2 = extract(afe, y2);
35
36 % Normalize sequences (per utterance)
37 mfccs1 = (mfccs1 - mean(mfccs1, 1)) ./ (std(mfccs1, 0, 1) + eps);
38 mfccs2 = (mfccs2 - mean(mfccs2, 1)) ./ (std(mfccs2, 0, 1) + eps);
39
40 fprintf('Comparing MFCC sequences of "%s" and "%s" using
    DTW...\n', file1, file2);
41
42 % Call the custom DTW function
43 try
44     dtw_dist_same = dtw_distance(mfccs1, mfccs2);
45     fprintf('DTW distance between same word (%s vs %s): %.4f\n',
        file1, file2, dtw_dist_same);
46
47 % Compare with a different word
48 file3 = 'no_1.wav'; % A different word from the dataset
49 [y3, fs3] = audioread(fullfile(audioFolderPath, file3));
50 if size(y3, 2) > 1; y3 = mean(y3, 2); end
51 y3 = y3 / max(abs(y3));

```

```

52     mfccs3 = extract(afe, y3);
53     mfccs3 = (mfccs3 - mean(mfccs3, 1)) ./ (std(mfccs3, 0, 1) +
        eps);
54
55     dtw_dist_diff = dtw_distance(mfccs1, mfccs3);
56     fprintf('DTW distance between different words (%s vs %s):
        %.4f\n', file1, file3, dtw_dist_diff);
57
58 catch ME
59     warning('DTW function (dtw_distance) not found or
        encountered error: %s. Please ensure it''s defined
        correctly.', ME.message);
60     fprintf('Conceptual DTW calculation not performed.\n');
61 end
62
63 % --- 3. Basic DTW-based Classifier (Conceptual/Small Scale) ---
64 fprintf('\n--- Basic DTW-based Classifier (Conceptual/Small
        Scale) ---\n');
65
66 % Data structure for DTW classification
67 dtw_train_data = {}; % Stores MFCC sequences
68 dtw_train_labels = {}; % Stores string labels
69
70 % Populate dtw_train_data for a few words
71 % Adapt these file names and labels to your actual dataset.
72 example_files_for_dtw = { ...
73     'yes_1.wav', 'yes_2.wav', 'yes_3.wav', ...
74     'no_1.wav', 'no_2.wav', 'no_3.wav' ...
75 };
76 example_labels_for_dtw = { ...
77     'yes', 'yes', 'yes', ...
78     'no', 'no', 'no' ...
79 };
80

```

```

81 fprintf('Preparing training data for DTW-based
    classification...\n');
82 for i = 1:length(example_files_for_dtw)
83     [y_dtw, fs_dtw] = audioread(fullfile(audioFolderPath,
        example_files_for_dtw{i}));
84     if size(y_dtw, 2) > 1; y_dtw = mean(y_dtw, 2); end
85     y_dtw = y_dtw / max(abs(y_dtw));
86     mfccs_dtw = extract(afe, y_dtw);
87     mfccs_dtw = (mfccs_dtw - mean(mfccs_dtw, 1)) ./
        (std(mfccs_dtw, 0, 1) + eps);
88
89     dtw_train_data{end+1} = mfccs_dtw; %#ok<AGROW>
90     dtw_train_labels{end+1} = example_labels_for_dtw{i};
        %#ok<AGROW>
91 end
92
93 % Test with one unknown sample
94 test_file_dtw = 'yes_4.wav'; % Assume this is an unseen 'yes'
95 [y_test, fs_test] = audioread(fullfile(audioFolderPath,
    test_file_dtw));
96 if size(y_test, 2) > 1; y_test = mean(y_test, 2); end
97 y_test = y_test / max(abs(y_test));
98 mfccs_test = extract(afe, y_test);
99 mfccs_test = (mfccs_test - mean(mfccs_test, 1)) ./
    (std(mfccs_test, 0, 1) + eps);
100
101 fprintf('Classifying test file "%s" using DTW...\n',
    test_file_dtw);
102 min_dist = inf;
103 predicted_label = 'unknown';
104
105 try
106     for i = 1:length(dtw_train_data)

```

```

107         current_dist = dtw_distance(mfccs_test,
            dtw_train_data{i});
108         if current_dist < min_dist
109             min_dist = current_dist;
110             predicted_label = dtw_train_labels{i};
111         end
112     end
113     fprintf('Predicted label for "%s": %s (Min DTW distance:
        %.4f)\n', test_file_dtw, predicted_label, min_dist);
114 catch ME
115     warning('DTW function not available for classification: %s',
        ME.message);
116     fprintf('DTW-based classification skipped.\n');
117 end
118
119 fprintf('\nLab Task 3 Complete. Remember to review the DTW
        implementation and limitations.\n');
120
121 % --- CUSTOM DTW FUNCTION (Include this at the end of
        lab_task_3.m or in a separate file) ---
122 function dist = dtw_distance(s, t)
123 % dtw_distance: Computes the Dynamic Time Warping (DTW) distance
        between two feature sequences.
124 %     s: First feature sequence (matrix of size [numFrames_s x
            numCoeffs])
125 %     t: Second feature sequence (matrix of size [numFrames_t x
            numCoeffs])
126 %     dist: The DTW distance.
127 %
128 % This is a simplified basic DTW implementation for pedagogical
        purposes.
129 % For highly optimized or constrained DTW, consider specialized
        toolboxes.
130

```

```

131     n = size(s, 1); % Number of frames in sequence s
132     m = size(t, 1); % Number of frames in sequence t
133
134     % Initialize cost matrix (D) with infinity
135     D = zeros(n+1, m+1);
136     D(1, :) = inf;
137     D(:, 1) = inf;
138     D(1, 1) = 0; % Start cost at (0,0)
139
140     % Compute the accumulated cost matrix
141     for i = 1:n
142         for j = 1:m
143             % Euclidean distance between feature vectors s(i,:)
144             % and t(j,:)
145             cost = norm(s(i,:) - t(j,:));
146
147             % Accumulate cost from the minimum of the three
148             % adjacent cells
149             D(i+1, j+1) = cost + min([D(i, j+1), D(i+1, j), D(i,
150                                     j)]);
151         end
152     end
153
154     % The DTW distance is the cost at the bottom-right corner of
155     % the accumulated cost matrix
156     dist = D(n+1, m+1);
157 end

```

Listing 3: MATLAB Code for Lab Task 3: Introduction to DTW

Questions for Lab Task 3

- How does the DTW distance reflect the similarity or dissimilarity between the two speech utterances?

- What are the computational challenges of scaling a pure DTW-based classifier to a large vocabulary or continuous speech?
- When would DTW be preferred over a simple aggregated feature approach (as in Lab Task 2)?

Lab Task 2: Enhancing the System (Conceptual)

Goal

To introduce more advanced concepts for handling temporal variability, larger vocabularies, and continuous speech in speech recognition. This task is primarily theoretical and aims to broaden your understanding beyond simple classification.

Procedure

[label=0.] **Introduction to Hidden Markov Models (HMMs):**

1.
 - Explain the limitations of simple classifiers and DTW for continuous speech or larger vocabularies.
 - Introduce the concept of Hidden Markov Models (HMMs) as a probabilistic framework for modeling sequential data like speech.
 - Briefly explain the key components of an HMM: states, observations, transition probabilities, and emission probabilities.
 - Discuss the two main problems solved by HMMs in speech recognition:
 - **Evaluation Problem (Forward Algorithm):** Given an HMM and an observation sequence, calculate the probability of that sequence.
 - **Decoding Problem (Viterbi Algorithm):** Given an HMM and an observation sequence, find the most likely sequence of hidden states.
 - **Learning Problem (Baum-Welch Algorithm):** Given an HMM and an observation sequence, adjust the HMM parameters to maximize the probability of the sequence.

2. Conceptual HMM-based Speech Recognition Pipeline:

- Discuss how HMMs are typically used in a complete ASR system:
 - Training separate HMMs for each word or phoneme.
 - Using the Viterbi algorithm for decoding (finding the most likely sequence of words given an observation sequence).
 - The role of a language model.

3. Brief Introduction to Deep Learning for Speech Recognition (Optional):

- Mention the paradigm shift towards deep learning (e.g., Recurrent Neural Networks - RNNs, Long Short-Term Memory - LSTMs, Transformers) for modern ASR systems.
- Explain how these models can learn complex temporal patterns directly from sequential MFCC features, often outperforming traditional methods.
- Highlight key deep learning architectures relevant to speech (e.g., End-to-End ASR).

Questions for Lab Task 4

- How do HMMs address the variability in speech duration and pronunciation compared to methods like KNN or simple DTW?
- What are the main challenges of building a complete, high-performance speech recognition system compared to the basic labs you performed?
- Why have deep learning approaches become dominant in advanced speech recognition, and what advantages do they offer over traditional methods?
- If you were to extend this project, what would be your next step (e.g., using delta/delta-delta features, more complex classifiers, a larger dataset)?

Lab 14

Speech Classification using Various Machine Learning Algorithms

December 23, 2025

December 20, 2025

1. Introduction to Speech Classification and MFCCs

Speech classification systems aim to enable computers to understand spoken language, a task crucial for various applications ranging from voice assistants to hands-free control systems. This lab explores the fundamental concepts and practical implementation of speech classification, focusing on the use of **Mel-Frequency Cepstral Coefficients (MFCCs)** as robust features. MFCCs are widely used in speech processing due to their ability to effectively represent the spectral envelope of speech, which is important for distinguishing different phonemes and words.

- **Objective:** To implement a basic speech Classification system capable of identifying spoken words or phonemes using MFCCs as features, and to compare the performance of different classification algorithms for this task.

2. Prerequisites

Before embarking on the implementation, ensure you have the following foundational knowledge and resources:

- **Digital Signal Processing Fundamentals:** A solid understanding of concepts such as sampling, Fast Fourier Transform (FFT), and various filtering techniques is essential for processing speech signals.
- **MFCC Feature Extraction:** Prior experience and familiarity with the process of extracting MFCC features from raw audio signals are crucial. This lab assumes you have already performed or understand this step.
- **Machine Learning Algorithm Basics:** A foundational knowledge of common classification algorithms, including **K-Nearest Neighbors (KNN)**, **Support Vector Machines (SVM)**, **Naive Bayes**, and **Decision Trees**, will be beneficial.
- **Programming Environment:** **MATLAB** with relevant toolboxes (e.g., Audio Toolbox, Statistics and Machine Learning Toolbox) is the designated programming environment.

- **Speech Dataset:** A small, labeled speech dataset is required. This could include isolated digits (e.g., "one", "two", "three"), or a few command words (e.g., "yes", "no", "up", "down"). Organize these audio files within an `audio/` folder in your MATLAB directory.

3. Core Components and Implementation Steps

The implementation of the speech Classification system will involve the following key stages, with a strong focus on the classification aspect:

3.1. Speech Data Acquisition and Preprocessing

- **Loading Audio Files:** Load the speech files from your `audio/` directory.
- **Pre-emphasis:** Apply a pre-emphasis filter to enhance higher frequencies.
- **Framing and Windowing:** Divide the speech signal into short, overlapping frames and apply a windowing function (e.g., Hamming window) to each frame.

3.2. MFCC Feature Extraction and Visualization

- **FFT Calculation:** Compute the Fast Fourier Transform (FFT) for each framed segment.
- **Mel Filter Banks:** Apply a set of Mel-scaled filter banks to the magnitude spectrum.
- **Logarithmic Energy:** Calculate the logarithm of the energy in each filter bank.
- **Discrete Cosine Transform (DCT):** Apply the Discrete Cosine Transform to the log energies to decorrelate the filter bank outputs and obtain the MFCCs.
- **Feature Visualization:** Visualize the extracted MFCC features (e.g., as a spectrogram-like plot or a 2D representation of the coefficients over time) to understand their structure and variability across different words.

3.3. Feature Preparation and Normalization for Machine Learning

- **Feature Vector Formation:** For each audio file, create a single feature vector by concatenating or averaging the MFCCs over time, or by creating a statistical summary (e.g., mean, variance) of the MFCCs.
- **Labeling:** Associate each feature vector with its corresponding word label.
- **Data Splitting:** Divide the dataset into training and testing sets to evaluate model generalization.
- **Feature Normalization:** Apply normalization techniques (e.g., z-score normalization, min-max scaling) to the MFCC features. This is crucial for many machine learning algorithms to prevent features with larger scales from dominating the learning process and to improve convergence.

3.4. Classification using Various Machine Learning Algorithms and Performance Comparison

This is the central focus of the lab. You will implement and evaluate several classification algorithms for isolated word Classification.

- **K-Nearest Neighbors (KNN):**
 - **Implementation:** Train a KNN classifier on the normalized MFCC features. Experiment with different values of k .
 - **Evaluation:** Calculate accuracy, precision, recall, and F1-score.
- **Support Vector Machines (SVM):**
 - **Implementation:** Train an SVM classifier. Experiment with different kernel functions (e.g., linear, RBF) and hyperparameters (e.g., C , gamma).
 - **Evaluation:** Calculate accuracy, precision, recall, F1-score, and generate a confusion matrix.
- **Naive Bayes Classifier:**

- **Implementation:** Train a Gaussian Naive Bayes classifier.
- **Evaluation:** Calculate accuracy, precision, recall, F1-score, and generate a confusion matrix.
- **Decision Trees:**
 - **Implementation:** Train a Decision Tree classifier. Experiment with parameters like maximum depth or minimum leaf size.
 - **Evaluation:** Calculate accuracy, precision, recall, F1-score, and generate a confusion matrix.
- **Performance Comparison (Advanced Metrics):**
 - **Metrics:** For each algorithm, compute and compare the following performance metrics:
 - * **Accuracy:** Overall correctness.
 - * **Precision:** Proportion of true positive predictions among all positive predictions.
 - * **Recall (Sensitivity):** Proportion of true positive predictions among all actual positives.
 - * **F1-score:** Harmonic mean of precision and recall, providing a balanced measure.
 - * **Confusion Matrix:** Visualize the performance of a classification algorithm, showing correct and incorrect predictions for each class.
 - * **ROC Curve and AUC (if applicable for binary classification or multi-class extensions):** To evaluate the classifier's ability to distinguish between classes.
 - **Discussion:** Analyze the strengths and weaknesses of each algorithm based on these metrics. Discuss which algorithm performs best for your specific dataset and why.

3.5. Introduction to Sequence-Based Comparison (Dynamic Time Warping - DTW)

- **Conceptual Understanding:** Understand the concept of **DTW** as a method for measuring the similarity between two temporal sequences that may vary in speed or duration.
- **Basic Implementation (Optional/Conceptual):** If time permits, implement a simple DTW algorithm to compare two MFCC sequences and observe its ability to align similar-sounding words even with temporal distortions.

3.6. Conceptual Overview of Advanced Techniques

- **Hidden Markov Models (HMMs):** Understand the probabilistic framework of HMMs for modeling sequential data like speech, and their role in early state-of-the-art speech Classification systems.
- **Deep Learning in Speech Classification:** Gain a conceptual understanding of how deep neural networks (e.g., Recurrent Neural Networks - RNNs, Long Short-Term Memory - LSTMs, Convolutional Neural Networks - CNNs) have revolutionized speech Classification by automatically learning hierarchical features and handling long-range dependencies in speech.

4. Expected Outcomes

Upon completion of this lab, you should be able to:

- Extract and visualize MFCC features from speech signals.
- Prepare and normalize speech features for machine learning algorithms.
- Implement and evaluate various machine learning classification algorithms (KNN, SVM, Naive Bayes, Decision Trees) for isolated word Classification.
- Compare the performance of these algorithms using advanced metrics such as accuracy, precision, recall, F1-score, and confusion matrices.

- Understand the fundamental concepts behind sequence-based comparison using DTW.
- Have a conceptual understanding of more advanced speech Classification techniques like HMMs and Deep Learning.

Questions for Lab Task

- Compare the overall accuracy of KNN, SVM (with different kernels), Naive Bayes, and Decision Tree classifiers. Which algorithm performed best and why do you think this is the case for your dataset?
- Analyze the per-class precision, recall, and F1-scores. Are there certain words that are harder to recognize than others? Why might this be? (e.g., acoustic similarity, limited training data for that word).
- Examine the confusion matrices for each classifier. What insights do they provide about the specific misclassifications made by each model? For example, is "up" often confused with "down"?
- Discuss the trade-offs between the complexity of each algorithm and its performance. Which algorithm offers a good balance for this specific task?

Results and Discussion

This section should present the outcomes of your experiments and your interpretations.

Performance Comparison of Classifiers

Discuss:

- Which classifier performed best overall (highest accuracy/F1-score)?
- How did the choice of k for KNN or kernel for SVM affect performance?
- Are there noticeable differences between precision, recall, and F1-score for individual classes? What does this imply about the classifier's strengths and weaknesses for specific words?

- Analyze the confusion matrices to discuss common misclassifications. For example, if "up" is often mistaken for "down", what acoustic similarities might explain this?
- Based on your results, which algorithm would you recommend for this specific isolated word Classification task, and why?

Limitations and Future Work

Identify the limitations of your current system:

- Small Dataset Size: Discuss how a larger, more diverse dataset could improve generalization.
- Isolated Word Classification: Explain the challenges of extending this system to continuous speech.
- Basic Features: Mention the potential benefits of adding delta and delta-delta MFCCs, or other feature types.
- Simple Classifiers: Discuss the potential for using more complex models like HMMs or deep neural networks for higher performance.

Table 1: Summary of Classifier Performance Metrics (Example)

Classifier	Accuracy (%)	Avg. Precision (%)	Avg. Recall (%)	Avg. F1-score (%)
KNN (K=1)	75.00	73.50	75.00	74.00
KNN (K=3)	80.00	78.20	80.00	79.00
SVM (Linear)	70.00	68.00	70.00	69.00
SVM (RBF)	85.00	84.50	85.00	84.70
Naive Bayes	60.00	55.00	60.00	57.00
Decision Tree	72.00	71.00	72.00	71.50

Note: Replace these example values with your actual experimental results.